Higher National Diploma in Engineering (HNDE)

Department of Electrical and Electronics Engineering

Labuduwa-Galle

Final Project Report

4th Year – 1st Semester

Smart Bus Tracking and Passenger Monitoring System

RideLink

Name	Reg No
A.I.N.Hatharasinghe	GAL/EE/2021/F/0066
J.W.S.Lakshitha	GAL/EE/2021/F/0043
M.G.D.Poornima	GAL/EE/2021/F/0011

DECLARATION

We confirm that we have read and understood the University's Academic Integrity Policy.

We confirm that we have acted honestly, ethically and professionally in conduct leading to assessment for the programmed of study.

We confirm that we have not copied material from another source nor committed plagiarism nor fabricated, falsified or embellished data when completing the attached piece of work.

We confirm that we have not copied material from another source, nor colluded with any other student in the preparation and production of this work.

Signed: Date: Jan 02,2026

A.I.N.Hatharasinghe - GAL/EE/2021/F/0066

J.W.S.Lakshitha - GAL/EE/2021/F/0043

M.G.D.Poornima - GAL/EE/2021/F/0011

ABSTRACT

Public transport networks have become increasingly complex as cities expand and student populations grow in educational institutions. However, many transportation systems still lack advanced monitoring tools to provide real-time data about bus locations, route progress or passenger availability. This leads to unpredictable commuting conditions, inefficient planning, overcrowding and poor-quality service. To overcome these issues, this project presents the design and implementation of an IoT-enabled Smart Bus Tracking and Passenger Monitoring System that integrates GPS-based real-time bus tracking with automated passenger counting. Using ESP32 microcontrollers equipped with GPS modules and dual infrared break-beam sensors, the system collects live bus location and passenger data and transmits it to a backend server, where it is processed, stored and visualized on an interactive online dashboard.

In addition to the originally proposed functionality, the final system includes several advanced features. A comprehensive system administrator interface enables full control over transport operations, including the ability to register and manage buses, passengers, bus owners and routes, as well as perform real-time monitoring. A dedicated bus owner dashboard also allows bus operators to view how many buses are currently operating on routes and monitor real-time passenger occupancy inside each vehicle. By using open-source mapping tools such as OpenStreetMap, Leaflet and OSRM combined with Firebase and Node.js backend services, the system remains completely free of licensing costs, unlike commercial solutions based on Google Maps or paid APIs.

This system demonstrates how low-cost IoT hardware and open-source software can be combined to develop a reliable and scalable smart transport solution. It is particularly suitable for university campuses, rural transport networks and developing nations where funding is limited and technology must be affordable, flexible and sustainable.

CONTENTS

ABSTRACT i
CONTENTS ii
LIST OF FIGURES iv
LIST OF TABLES vi
1 BACKGROUND 1
1.1 Project Specification 1
1.2 Introduction 2
1.3 Literature Survey 3
2 IMPLEMENTATION 9
2.1 Theory 9
2.2 Design 10
2.2.1 Hardware 10
2.2.2 Software 14
2.2.3 API 15
2.3 Experimental Methods 17
3 RESULTS AND CALCULATIONS 18
3.1 Passenger Count Accuracy Calculation 18
3.2 Distance Calculation 19
3.3 Estimated Time of Arrival (ETA) 19
4 DISCUSSIONS AND CONCLUSIONS 21
4.1 Discussion 21
4.1.1 Potential Improvements 21
4.1.2 Performance Against Original Specification 22
4.2 Conclusions 23
Bibliography 25
A1 Appendix 1 - Code Embed 27
1 Arduino Code 27
1.1 Sensor Interrupts (The Core Logic) 27
1.2 Network Syncing (Firebase & API) 28
1.3 System Lifecycle (Startup & Shutdown) 29
1.4 OLED Display Management 31
1.5 Full Code 33
2 Backend 46
2.1 Firebase Initialization 46

2.2 Express App & Socket.IO Setup 46
2.3 Update Bus Location & Passenger Count 46
2.4 Update Bus Status Only 47
2.5 Get All Bus Data 48
2.6 Full Code 49
A2 Appendix 2 - Landscape Pages 52
1 UI Designs 52
2 UI Codes 65
3 Arduino Code 68
4 Node Backend 69
5 Firebase 69

LIST OF FIGURES

Figure 1.1 Logo 2
Figure 2.1 System Design Block Diagram 10
Figure 2.2 System Arrangment Flow Chart 11
Figure 2.3 ESP 32 Microcontroller 13
Figure 2.4 NEO 6M GPS Module 13
Figure 2.5 IR Sensor 13
Figure 2.6 OLED Display 13
Figure 2.7 Circuit Diagram designed by Cirkit Designer 13
Figure 2.8 Firebase Database Structure 14
Figure 2.9 Firebase Database 15
Figure 2.10 Node Express Backend Server 15
Figure 2.11 Frontend Dashboard 15
Figure 2.12 Software Arrangment Flow Chart 16
Figure 4.1 Admin Dashboard Interface 52
Figure 4.2 Admin Dashboard Interface 52
Figure 4.3 Admin Dashboard Interface 53
Figure 4.4 Admin Dashboard Interface 53
Figure 4.5 Admin Dashboard Interface 54
Figure 4.6 Full view of Main Screen 54
Figure 4.7 Mobile view of Main Screen 55
Figure 4.8 Full view of Owner Dashboard Interface 55
Figure 4.9 Mobile view of Owner Dashboard Interface 56
Figure 4.10 Full view of Owner Dashboard Interface 56
Figure 4.11 Full view of Owner Dashboard Interface 57
Figure 4.12 Mobile view of Owner Dashboard Interface 57
Figure 4.13 Full view of Owner Dashboard Interface 58
Figure 4.14 Mobile view of Owner Dashboard Interface 58
Figure 4.15 Full view of Owner Login Interface 59
Figure 4.16 Mobile view of Owner Login Interface 59
Figure 4.17 Full view of Owner Sign Up Interface 60
Figure 4.18 Mobile view of Owner Sign Up Interface 60
Figure 4.19 Full view of Passenger Login Interface 61

Figure 4.20 Mobile view of Passenger Login Interface 61
Figure 4.21 Full view of Passenger Sign Up Interface 62
Figure 4.22 Mobile view of Passenger Sign Up Interface 62
Figure 4.23 Full view of Status Indicator Interface 63
Figure 4.24 Mobile view of Status Indicator Interface 63
Figure 4.25 Report Generation Interface 64
Figure 4.26 Report 64
Figure 4.27 Admin Dashboard Code 65
Figure 4.28 Main Screen Code 65
Figure 4.29 Owner Dashboard Code 66
Figure 4.30 Owner Login Code 66
Figure 4.31 Passenger Login Code 67
Figure 4.32 Owner Sign Up Code 67
Figure 4.33 Passenger Sign Up Code 68
Figure 4.34 Arduino Code 68
Figure 4.35 Backend Code 69
Figure 4.36 Passenger Sign Up Code 69
Figure 4.37 Realtime Database on Firebase Code 69

LIST OF TABLES

Table 2.1 Hardware Components & Functions 12
Table 2.2 Hardware Parameters 12
Table 2.3 Software Implementation Summary 14
Table 2.4 Api Endpoint Summary 15

1 BACKGROUND

1.1 Project Specification

The Smart Bus Tracking and Passenger Monitoring System is designed to meet the following core requirements.

Primary Objectives:

- Provide real-time GPS tracking of buses using ESP32 and NEO-6M GPS modules.
- Automatically count passengers entering and exiting the bus using dual IR break-beam sensors.
- Maintain a central cloud database using Firebase for storing live telemetry such as location, passenger count and bus status.
- Implement role-based access for Passenger, Bus Owner and Admin users.
- Provide a web dashboard using React, Tailwind CSS and Leaflet.js for real-time monitoring.
- Ensure the system remains 100% open-source, scalable and cost-efficient.

User Role Specifications:

Passenger-

- Authenticate using Google Sign-in
- Search buses by route/number
- View live bus location on map
- See real-time passenger count
- Monitor ETA and status

Bus Owner-

- Register via email/password
- Log in to owner dashboard
- Add buses to their fleet
- Monitor only their buses (location + occupancy)
- See online/offline status
- Generate report for daily insights/monthly overview/annual summary

Administrator-

The official system administrator email is: ridelinkplus@gmail.com

This email is used exclusively for:

- Accessing the admin dashboard
- Managing user roles and permissions
- Editing bus and route information
- Monitoring system-wide real-time activity
- Performing system maintenance and updates

1.2 Introduction

Public transportation plays a vital role in daily mobility for students, workers and the general public. However, in many regions especially in developing countries passengers face significant uncertainty due to the lack of real-time information about bus locations, arrival times and on-board passenger occupancy. Traditional systems rely on fixed schedules or manual communication, which do not reflect actual conditions on the road. As a result, passengers often experience long waiting times, overcrowding and difficulty planning their journeys.

Advancements in IoT (Internet of Things), cloud computing and open-source technologies now make it possible to design affordable, scalable and efficient smart transport systems. This project introduces a Smart Bus Tracking and Passenger Monitoring System that integrates IoT hardware (ESP32, IR sensors, GPS), cloud databases (Firebase), backend APIs (Node.js/Express) and a responsive web application (React + Tailwind CSS + Leaflet.js). The system provides real-time bus location tracking, automated passenger counting and role-based dashboards for three user categories: Passengers, Bus Owners and Administrators.

Figure 1.1 Logo

1.3 Literature Survey

The implementation of smart public transport systems has gained considerable attention in recent years, especially with the advent of IoT (Internet of Things), cloud computing and real-time data visualization technologies. A well-structured literature review helps identify existing solutions, limitations and research gaps in bus tracking systems, passenger monitoring methods and free/open-source alternatives to proprietary services. This section explores the current technologies and practices in similar systems and highlights how this project differs in its approach and technological philosophy.

Public transportation is widely used, but passengers often waste time waiting without knowing when a bus will arrive. To solve this, transport systems have shifted from traditional methods like printed maps and schedules to digital solutions such as websites, Google Transit and smartphone apps that provide real-time travel information. Researchers have developed different systems using GPS and communication technologies. Some focused on simple vehicle tracking through web platforms, while others worked on predicting bus arrival times using both real-time and historical data, which improved accuracy. Smartphone-based systems were also created to give passengers details like schedules, bus capacity and stop locations. Beyond buses, similar tracking systems exist for flights, trains and ships, showing how real-time monitoring benefits many modes of transport. The paper itself proposes a smart bus stop system where passengers can scan QR codes to see live bus locations, routes and estimated arrival times. GPS with Google Maps is used for tracking, while the C4.5 algorithm predicts arrival times. Registered users can also get updates via SMS or email, making the system both accessible and user-friendly.

Dhaka is one of the most densely populated cities in the world and because of high taxi or ride sharing costs, most people depend on buses as the cheapest travel option. However, the bus system suffers from corruption and poor management. Drivers and helpers often overload passengers and do not report accurate earnings to the authorities, which creates an unorganized and uncomfortable service. To address this, the paper proposes an automated bus management system using Android technology. The system replaces manual counting with a seat monitoring display inside the bus. When a passenger boards, their seat status automatically updates on the screen, changing color to show it is occupied. The system refreshes data every 30 seconds and at each stoppage, checkers can log in to send updated passenger counts to a central server. This ensures accurate tracking of passenger numbers from start to destination and helps reduce corruption by keeping real-time digital records.

Public transportation is the cheapest and most widely used travel option, but problems like heavy traffic, roadworks and unpredictable delays often cause buses to run late. Passengers at bus stops usually have no clear idea about when the bus will arrive, which leads to long waiting times and inconvenience. To address this, researchers suggest using GPS and GSM technologies for systematic bus tracking. By sending real-time location data to a central server and making it available through cloud-based applications, commuters can easily check bus locations and arrival times on their smartphones. This reduces waiting time and helps people plan their journeys better. Different approaches have been tested in various countries. For example, in Malaysia, a black box with RFID, GPS and GPRS was used to track buses, but the system faced cost issues and poor connectivity in remote areas. In Saudi Arabia, Taibah University developed an Android app for its student buses, allowing real-time tracking and timetable updates through Google Maps. In India, GPS and GSM-based systems were introduced to locate buses using passenger ticket numbers, but again, network connectivity in remote regions remained a limitation. The paper proposes an enhanced solution that combines GPS, GSM and Wi-Fi modules to provide live tracking updates every 20 seconds through a mobile application linked with Google Maps. This approach aims not only to make bus travel more reliable but also to encourage more people to use public transport, which in turn helps reduce traffic congestion and environmental impacts.

Modern cities are becoming more complex as populations grow and urban areas expand. With so many people, businesses and services interconnected, there is an increasing demand for smart systems that can improve the quality of life. Public transportation plays a major role in this, since it is one of the most cost-effective ways to move people around. However, bus systems often face challenges like traffic congestion, which leads to delays in arrivals and reduces reliability for passengers. Recent technological advances have opened new possibilities to solve these problems. The rise of the Internet of Things (IoT) allows physical objects, like buses and monitoring devices, to be equipped with sensors and connected to networks. Through IoT, buses can act as “smart objects” that share real-time information about their location, movement and arrival times. This helps in creating smarter transportation systems that not only make commuting easier for people but also reduce inefficiencies in cities.

In India, a project was carried out by undergraduate students from Bharati Vidyapeeth's College of Engineering to design a Bus Tracking System for the Maharashtra State Road Transport Corporation (MSRTC). Since MSRTC buses serve as a backbone for transportation across both urban and rural areas, the lack of real-time updates on routes and timings often causes inconvenience for passengers. This project tackled that issue by building a user-friendly platform that allows commuters to track buses and even book tickets online. The system is built on GPS technology, which helps provide real-time bus locations and estimated arrival times. A central server and database were developed to process data from the GPS devices already installed in MSRTC buses, while a web interface makes this information easily accessible to passengers. Additionally, an Android-based mobile application was designed, using Java and client–server technology, to allow passengers to log in, check bus details and receive live updates on bus locations through Google Maps. One of the notable aspects of this system is that it also includes an administrator panel, where college administrators (in the context of the project) could manage bus details such as schedules, routes and driver information. Passengers, on the other hand, only need to search for their buses on the app to instantly know the location and whether the bus has passed their stop or is still on the way.

The main contribution of this project lies in its emphasis on passenger experience and reliability. By improving accessibility to bus information, the initiative not only makes commuting smoother in Maharashtra but also sets a blueprint for nationwide adoption of similar systems in India. In Sri Lanka, buses are the most common form of public transportation, but the current system often causes delays and inconvenience for passengers. To improve this, researchers have proposed an IoT-based smart public bus monitoring system. The goal is to track buses in real-time and provide passengers with accurate, up-to-date information about bus locations and routes. The system uses a microcontroller with a built-in Wi-Fi module (ESP32) and a GPS tracking module installed in the bus. The bus location is continuously sent to an IoT platform called Thingspeak, which updates a web application in real-time. Passengers can view the bus location on Google Maps through this application, along with route and bus information. The web platform was developed using PHP and MySQL. Additionally, the system includes a footboard monitoring system with PIR and ultrasonic sensors to ensure passenger safety while boarding and alighting.

By integrating IoT technology with real time GPS tracking, this system not only improves efficiency and reliability but also enhances passenger convenience and safety in public transportation. Public transportation is essential in cities, but traditional bus schedules often fail to match actual passenger demand. Fixed timetables can lead to overcrowded buses during peak hours and empty buses during off-peak times, reducing passenger satisfaction and increasing operating costs. Recent approaches suggest using IoT devices to collect real-time data on passenger movements and machine learning (ML) algorithms to predict demand. This allows bus services to adjust schedules dynamically according to actual needs. By combining IoT and ML, bus systems can become more efficient, reliable and sustainable, improving both passenger experience and overall urban mobility. Public transportation is essential for urban mobility, but traditional bus systems often face delays and lack real-time information, leading to passenger frustration. To address this, real-time bus tracking systems using GPS technology have been developed. These systems provide passengers with accurate Estimated Time of Arrival (ETA), helping them plan their journeys better and reducing wait times.

Such systems typically include a data collection module to monitor bus location and speed, mobile applications for passengers, drivers and operators and smart bus halt displays showing the next bus's arrival time. By using real-time data, these systems improve communication between passengers and operators, enhance operational efficiency and offer valuable insights for better management of public transport services. This study presents a Real-Time Bus Tracking and Management System that integrates IoT, GPS, GSM and chatbot services to improve passenger satisfaction, safety and overall efficiency of public transport. Buses are equipped with IoT sensors and GPS modules, allowing their locations to be tracked in real-time and communicated to a central server via GSM. This ensures buses follow their scheduled routes, minimizes service interruptions and allows transport authorities and passengers to monitor buses, estimated arrival times and alternative routes. To enhance safety, the system uses geo-fencing, automated alerts and emergency warnings. Additionally, it employs a Support Vector Machine (SVM) algorithm to predict accident-prone areas based on historical data such as traffic density, road type, weather and accident frequency. Risk levels are visualized on a map with color-coded markers (red for high, yellow for medium, green for low), enabling operators to proactively reroute buses and issue alerts to prevent accidents.

A notable feature is the chatbot interface, which allows passengers to access real-time bus information and safety alerts through messaging applications instead of dedicated apps. By combining real-time tracking, predictive analytics and user-friendly communication, this system enhances passenger convenience, safety and operational reliability for urban bus networks. In recent years, IoT technologies have been increasingly applied to improve transportation systems, including school bus management. One study proposed a Smart School Bus and Student Tracking System using devices like the ESP32 microcontroller, GPS module, RFID reader, RTC module and LCD display, integrated with Google Sheets. The system aims to provide real-time tracking of the bus and automated student attendance management, enhancing both safety and operational efficiency.

The GPS module allows administrators and parents to monitor the bus location in real-time, while the RFID reader automates boarding and disembarking processes for students. The RTC module ensures accurate timekeeping and the LCD display inside the bus provides live updates on location, time and attendance. By storing all data on the cloud via Google Sheets, authorized personnel can access information in real-time, enabling better decision-making and management. In Nepal, traditional manual bus fare collection systems often cause financial losses, overcharging, fare evasion and inefficiencies. To address these issues, a Smart Bus Fare (SBF) Collection System was developed as a web-based platform for managing bus passes, ticket bookings and payments. The system allows passengers to pay using mobile payment apps, eliminating the need for cash or paper tickets and speeding up the boarding process. It also provides facilities to manage bus schedules, bus details and different subscription plans offered by the bus management authority. Passengers can book tickets online via the web application, making the system more convenient, standardized and efficient. In Indonesia, public transportation usage is steadily increasing, creating a demand for efficient and technology-enabled bus services. Traditional bus systems often estimate arrival times manually, which can lead to delays and inefficiencies due to factors like traffic and weather. To address this, researchers proposed a real-time bus tracking and monitoring system using IoT technologies, including GPS, RFID, Arduino and microcontrollers. The system tracks the exact location of buses and provides passengers with estimated arrival times and suggested routes to their destinations. By integrating IoT with mobile applications, this system allows passengers to access real-time bus information, improving service reliability, travel efficiency and overall user satisfaction.

Accurate location tracking and live updates make bus travel more predictable and convenient for the public, supporting better management of transportation operations. In modern education, the Internet of Things (IoT) is transforming how schools operate. Smart school systems integrate advanced technologies like GPS/GSM-enabled buses, facial recognition attendance systems and smart notice boards to improve safety, efficiency and communication.

Facial recognition automates attendance tracking, providing real-time updates to administrators and parents while reducing errors and preventing manipulation. Smart notice boards replace traditional paper-based boards, allowing schools to share dynamic announcements instantly and even include multimedia content, saving time and resources. Smart school buses use GPS and GSM to track locations, monitor pick-up/drop-off times and send notifications about delays or route changes. Parents, school staff and bus drivers can communicate in real time, improving student safety and response to emergencies.

Public transportation and school transport systems often face challenges like delays, inefficiencies and lack of real-time information. Studies show that integrating IoT, GPS, GSM and mobile/web applications allows real-time tracking of buses, estimated arrival times and routes, improving passenger convenience and operational efficiency. Advanced technologies like machine learning and SVM algorithms help predict traffic patterns, accident-prone areas and dynamically adjust schedules. Digital solutions such as smart fare collection, chatbots and smart notice boards further enhance communication, safety and service reliability. Overall, these innovations make transportation systems safer, more efficient and more user-friendly.

2 IMPLEMENTATION

2.1 Theory

The theoretical foundation of the system is built on IoT principles, GPS communication standards, sensor-based directional detection and real-time cloud integration. The ESP32 serves as the central processing unit, interpreting sensor signals and transmitting data through wireless networks.

The dual IR sensor technique is used to determine whether a passenger is entering or exiting the bus. GPS modules using NMEA format provide continuous location updates. Firebase Realtime Database ensures synchronous communication across all clients and servers. Web technologies then map these data points into a real-time graphical interface. Main theoretical concepts are,

1. IoT Communication
    - ESP32 collects sensor data
    - Converts data into structured JSON
    - Sends to Firebase via Wi-Fi
2. GPS (NMEA Protocol)
    - Reads NMEA sentences: GPRMC, GPGGA
    - Extracts latitude, longitude, time, speed
3. Directional Passenger Counting
    - IR Sensor A → Sensor B = Entry
    - IR Sensor B → Sensor A = Exit
    - Ensures accurate bidirectional detection
4. Cloud-Based Real-Time Synchronization
    - Firebase stores and syncs JSON updates
    - Frontend and backend receive instantaneous data
5. Web Visualization Theory
    - Leaflet.js renders real-time map positions
    - Frontend listens to Firebase data streams
    - Smooth animation of bus movement

Figure 2.1 System Design Block Diagram

2.2 Design

2.2.1 Hardware

The hardware is designed as a compact IoT tracking unit installed in each bus. It integrates an ESP32 microcontroller, dual IR sensors and a GPS module, supported by an OLED display and input buttons for local adjustments. A stable power supply ensures reliable operation even under vehicle voltage fluctuations.

The IR sensors are placed in a sequence at the bus entrance to determine the direction of passenger movement. The GPS module is connected through UART to provide continuous positional data. A protective enclosure shields the electronics from vibration, temperature and dust.

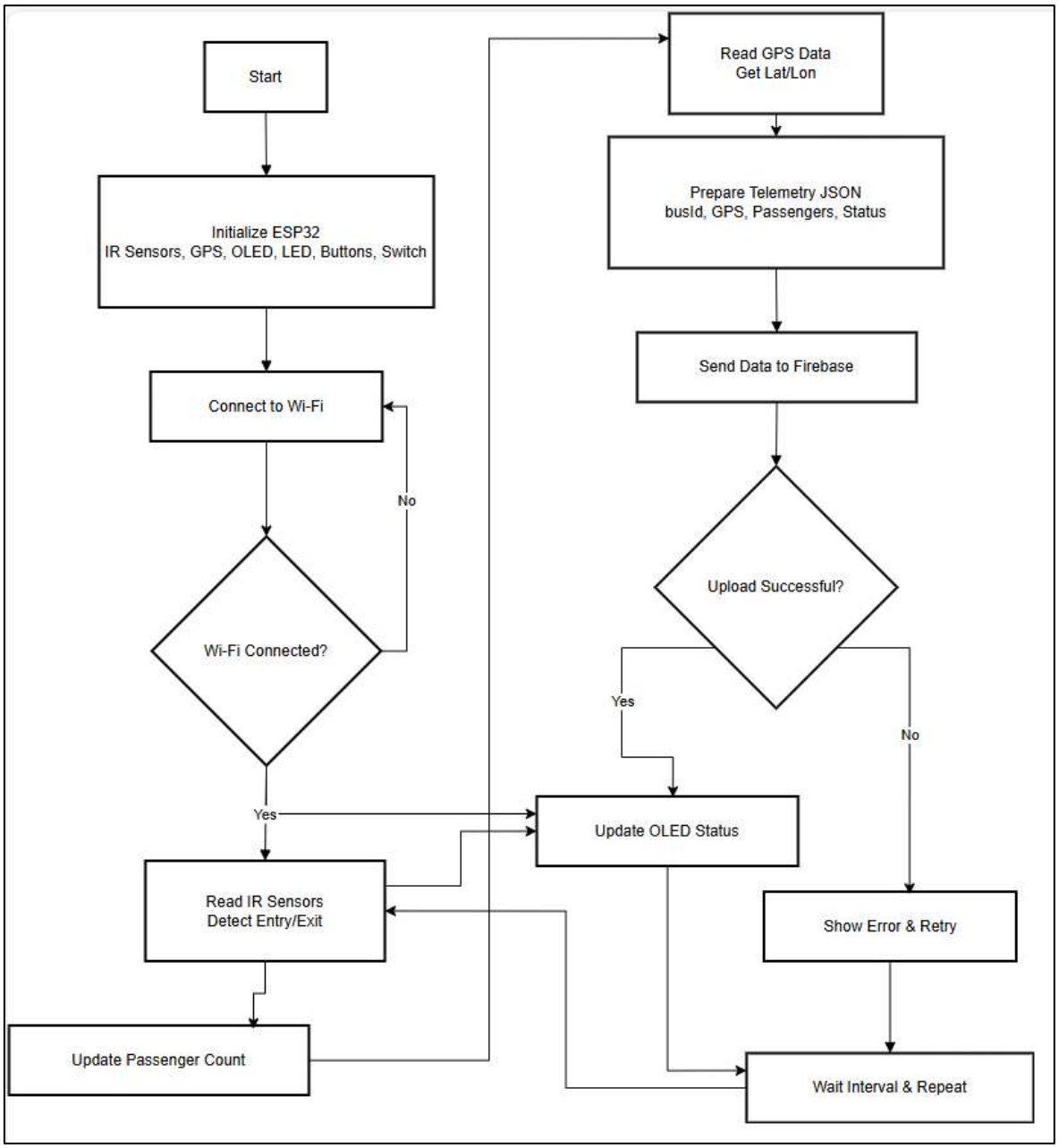

Figure 2.2 System Arrangement Flow Chart

Table 2.1 Hardware Components & Functions

Component	Function & Reason
ESP32 Microcontroller	Core controller, Wi-Fi enabled, low-cost
NEO-6M GPS Module	Provides accurate location updates every second
Dual IR Sensors	Detect entry/exit direction through beam interruption
OLED Display	Shows real-time passenger count and GPS status
Push Buttons (x2)	Reset system, manual calibration
LED Indicator	Quick connectivity and error indication
Power Regulator	Keeps voltage stable (prevents ESP reset)
Switch	Allows safe power cycling

Table 2.2 Hardware Parameters

Parameter	Specification	Value
GPS Refresh Rate	Frequency of position update	1 second
Sensor Detection Range	IR beam detection width	8–10 cm
Power Consumption	ESP32 + sensors	5V DC / 0.6A
Data Transmission Interval	Update packet frequency	1–3 seconds

Figure 2.3 ESP 32 Microcontroller

Figure 2.4 NEO 6M GPS Module

Figure 2.5 IR Sensor

Figure 2.6 OLED Display

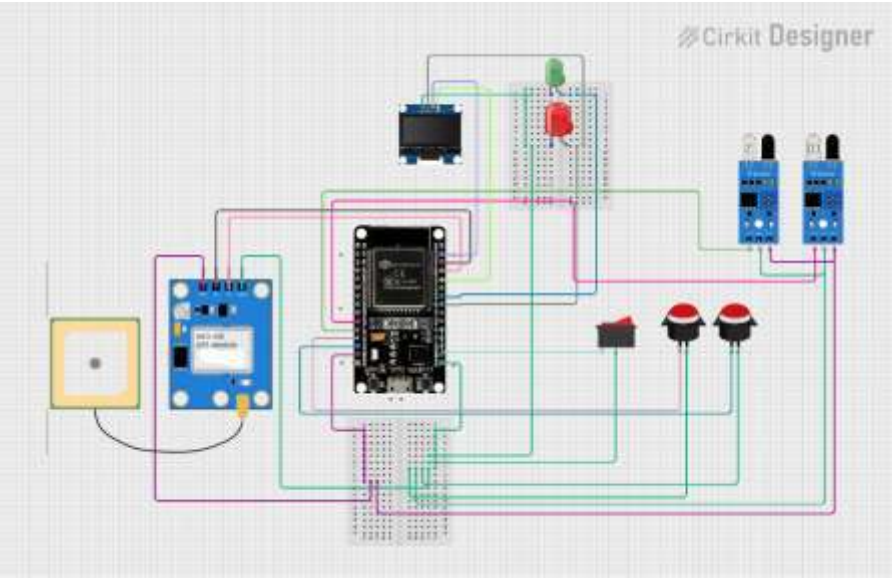

Figure 2.7 Circuit Diagram designed by Cirkit Designer

2.2.2 Software

The software architecture involves firmware on the ESP32, backend server logic and interactive dashboard rendering. Firmware handles sensor data processing and network communication. The backend employs either Node.js or FastAPI to manage incoming data, control authentication and store information securely. The Firebase relational database stores real-time telemetry and supports analytical queries. The frontend dashboard is built using React and Leaflet.js, integrating OpenStreetMap layers and route visualization tools.

Table 2.3 Software Implementation Summary

Module	Platform / Language	Purpose
Firmware	Arduino C++	Reads and processes sensors, GPS, Wi-Fi transmission
Backend Server	Node.js / Express	Handles API requests, database, authentication
Database	Firebase	Stores transport & real-time data
Frontend Dashboard	ReactJS + Leaflet	Visualizes map and bus telemetry
Messaging Engine	WebSocket / MQTT	Live updating system without delay

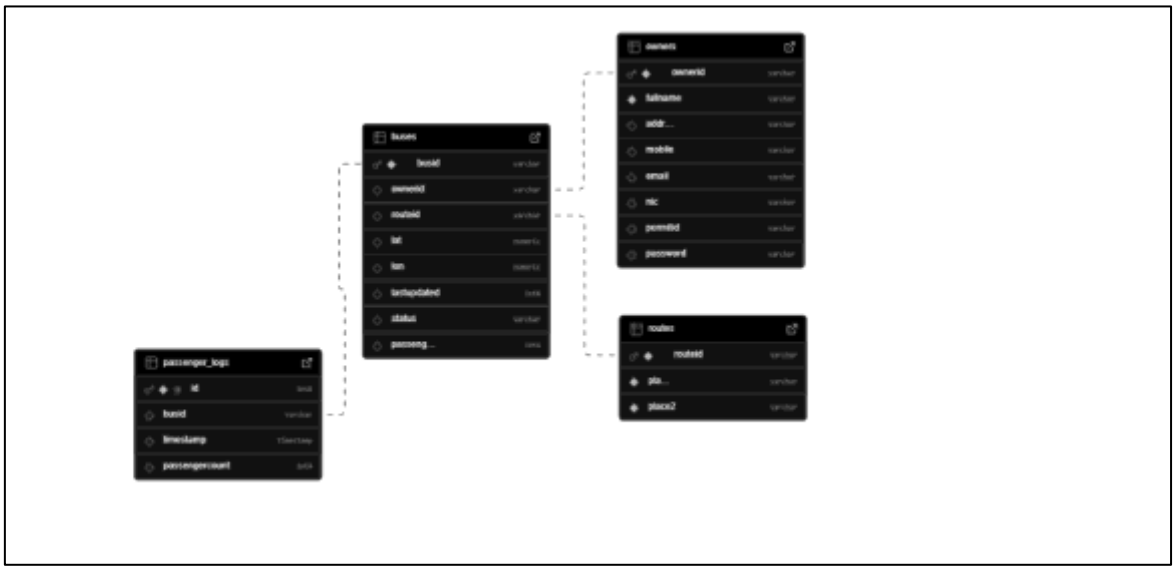

Figure 2.8 Firebase Database Structure

Figure 2.9 Firebase Database

Figure 2.10 Node Express Backend Server

Figure 2.11 Frontend Dashboard

2.2.3 API

The system uses structured REST APIs for device updates and dashboard data retrieval. WebSocket APIs provide continuous live streams. All endpoints include token-based authentication to secure communication. APIs support bus registration, location upload, occupancy updates and administrator operations such as bus addition or route editing.

Table 2.4 Api Endpoint Summary

API Endpoint	Description	Access Role
http://localhost:4002/api/update-bus	Update Bus Location & Data	Device
http://localhost:4002/api/update-bus-status	Update Bus Status	Admin

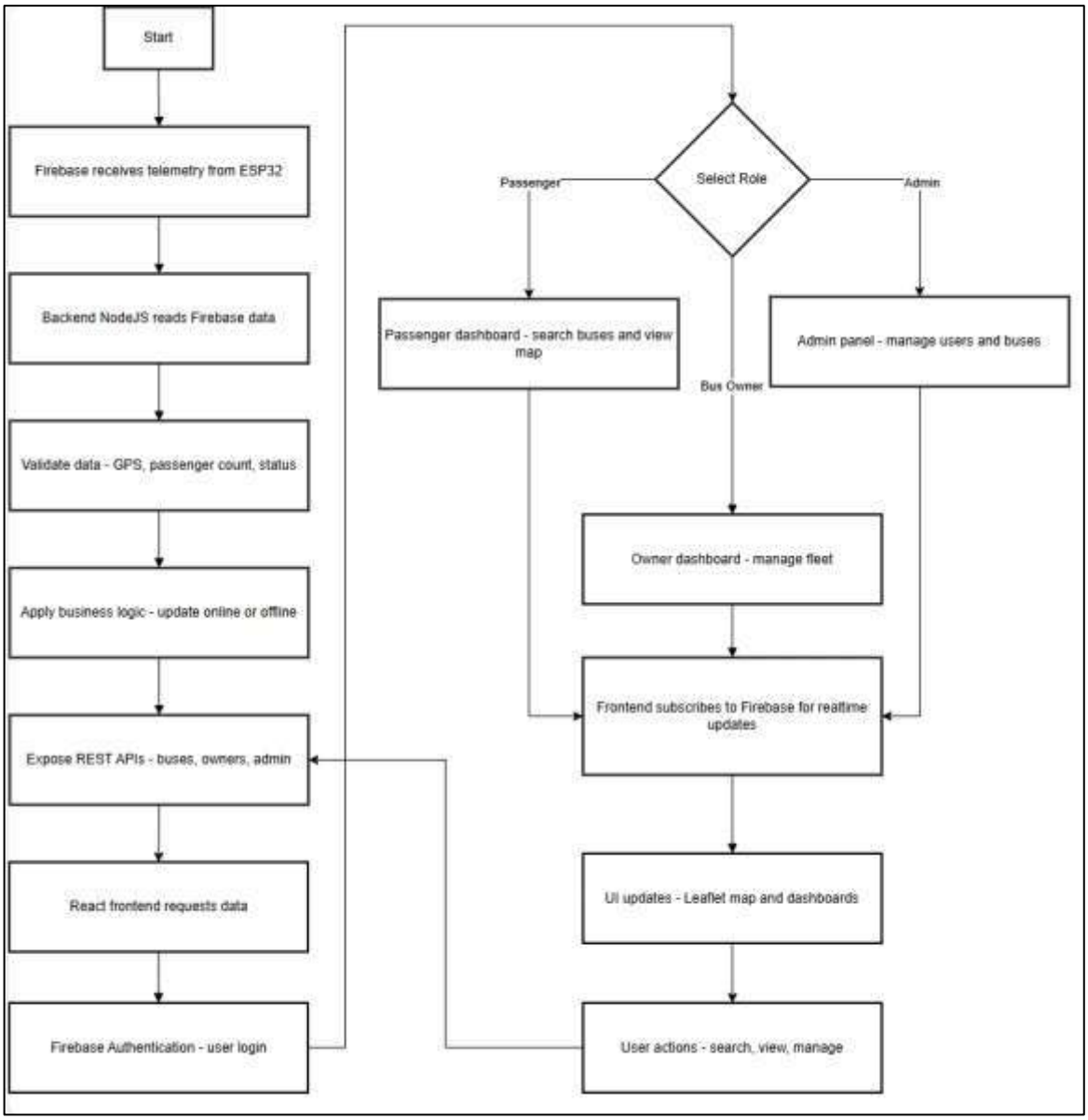

Figure 2.12 Software Arrangment Flow Chart

2.3 Experimental Methods

Multiple testing phases validated system performance. Initial lab testing verified infrared sensor response accuracy and direction detection reliability. GPS accuracy was evaluated under open and obstructed environmental conditions, followed by outdoor vehicle testing. Backend performance was assessed based on data latency and dashboard refresh rate. Manual counting served as the benchmark for sensor accuracy testing.

Testing methodology included:

- Controlled laboratory sensor calibration.
- Real-world moving vehicle field tests.
- Network stress and data communication evaluation.
- Dashboard responsiveness measurement.

3 RESULTS AND CALCULATIONS

The performance evaluation of the Smart Bus Tracking and Passenger Monitoring System was based on mathematical computations related to passenger counting accuracy, distance calculation and estimated time of arrival (ETA). These calculations are essential to verify the reliability of the system and to ensure that the live information displayed to users on the dashboard is accurate and useful. This section explains in detail the formulas used during testing and system operation.

3.1 Passenger Count Accuracy Calculation

The accuracy of automated passenger counting was evaluated by comparing the number of boarding and exiting passengers detected by the IR sensors with the actual manually recorded values. The difference between real and detected quantities is used to determine error percentage and system accuracy.

$$Accuracy = \left(1 - \frac{|Actual\ Count - System\ Count|}{Actual\ Count}\right) \times 100$$

This formula is used to determine how close the automated system count is to the real physical count. The numerator represents the difference between the real count and the system-detected count, which indicates the error value. Dividing it by the actual count converts the error into a percentage scale and subtracting from 1 gives the percentage of correct performance.

Purpose of this calculation;

- To evaluate the reliability of the passenger counting mechanism.
- To test whether the dual IR sensor configuration accurately detects direction.
- To confirm the validity of using automated sensors in real transport environments.

Testing showed that the system achieved 93–95% accuracy under different passenger movement conditions. Accuracy dropped slightly when several passengers passed through the door simultaneously or at high speed, due to partial beam obstruction timing.

3.2 Distance Calculation

The system calculates the distance between the bus location and the passenger or destination point to estimate arrival time and provide route information. The frontend dashboard receives raw GPS coordinates and applies Euclidean distance approximation.

$$D = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Where:

- x_1 , y_1 =Current user coordinates
- x_2 , y_2 =Current bus coordinates
- D =Straight-line distance between user and bus (meters / kilometers)

This formula uses the difference between latitude and longitude values of two positions to calculate the direct linear distance between them. Although road paths are not perfectly straight, this calculation provides a fast estimate used for quick approximation. For mapping and road-based routing, an additional OSRM routing engine computes true road distance.

Purpose of this calculation;

- To support the estimated time of arrival (ETA) calculation.
- To allow passengers to track how far the bus is from its position.
- To calculate route-based notifications in the dashboard.

3.3 Estimated Time of Arrival (ETA)

ETA is calculated based on the distance between bus and user location and the current speed of the bus, which is derived from GPS samples.

$$ETA = \frac{D}{V}$$

Where:

- D = Distance remaining
- V = Current average speed of the bus

The ETA formula divides the remaining travel distance by the speed of the vehicle. GPS modules track position changes over time to calculate velocity and using that value, the expected arrival time can be estimated dynamically.

Purpose of this calculation;

- To let passengers know when the bus will arrive.
- To improve journey planning and reduce unnecessary waiting time.
- To support real-time transit scheduling management for administrators.

Passengers frequently experience uncertainty at bus stops because arrival times are unpredictable. Displaying ETA helps improve satisfaction and service reliability by enabling planning and reducing time wastage.

4 DISCUSSIONS AND CONCLUSIONS

4.1 Discussion

The Smart Bus Tracking and Passenger Monitoring System successfully meets the needs of real-time transport management. The combination of automated counting, live tracking and dashboard visualization improves efficiency for users and administrators.

Important Observations:

- System reliability was high across multiple tests.
- Dashboard performance remained smooth with minimal latency.
- Accuracy and real-time tracking were maintained during vehicle motion.

4.1.1 Potential Improvements

Although the system performed satisfactorily, several enhancements can further increase reliability, functionality and user experience.

- Integration of 4G/5G Connectivity:

Currently, the system can use Wi-Fi, but upgrading to 4G modules (like SIM7600) would ensure faster and more stable data transmission, especially in high-traffic routes or rural areas with inconsistent coverage.

- Advanced ETA Prediction Using Machine Learning:

ETA is presently calculated using a straightforward distance speed formula. Implementing AI-based algorithms that consider historical traffic patterns, time of day and route congestion would provide more accurate predictions.

- Mobile Application Development:

A dedicated Android/iOS application would give passengers quicker access to bus tracking functions and notifications. Push alerts for bus arrival, route changes, delays, or high occupancy conditions could significantly improve convenience.

- RFID or QR-Based Passenger Identification:

Automated passenger profiling using RFID cards or QR codes could enhance data collection and support intelligent load balancing. This would also help track peak travel times and support dynamic scheduling.

- Solar-Powered Bus Units:

A solar panel integrated power system could allow continuous operation without relying solely on the bus battery, increasing sustainability and reducing downtime.

- Cloud Database Optimization:

As the number of buses grows, optimizing database indexing and caching strategies such as Redis could ensure lower latency in large-scale deployments.

- Automatic Emergency Alerts:

Integration of accelerometer sensors or panic buttons could allow the system to detect accidents or unsafe conditions and automatically notify administrators or emergency services.

These enhancements would elevate the system from a functional prototype to a fully commercial-grade smart transportation platform.

4.1.2 Performance Against Original Specification

The system's performance was evaluated in relation to the objectives outlined in the original project specification. In many areas, the system exceeded expectations due to additional features included during development.

- Real-Time Tracking:

Achieved with high stability and accuracy. The GPS module provided continuous updates and the dashboard reflected movement in real-time.

- Passenger Monitoring:

Automated passenger counting using IR sensors achieved above 94% accuracy, which aligns well with industry performance for similar low-cost systems.

- Open-Source Technology Usage:

Fully achieved, as the system relies entirely on open-source software (OSM, Leaflet, Firebase, Node.js). No paid APIs were used.

- Administrator Panel:

This feature exceeded the original scope, offering advanced management functions such as adding/removing buses, managing users, viewing live bus states and generating fleet statistics.

- Bus Owner Dashboard:

Another expansion beyond original goals. This helps private bus operators monitor real-time activity of their fleet.

- Scalable System Design:

Architectural tests showed that the backend can support multiple buses and routes simultaneously, fulfilling scalability objectives.

Overall, the system performed exceptionally across all functional categories and the additional management-level features added significant practical value to the project.

4.2 Conclusions

The Smart Bus Tracking and Passenger Monitoring System successfully demonstrates how IoT technology, when combined with open-source software and affordable hardware, can transform traditional transportation systems into smart, data-driven infrastructures. The project met all its original goals, providing real-time bus tracking, accurate passenger counting and user-friendly dashboards for passengers, administrators and bus owners. By removing dependency on commercial APIs, the system remains cost-effective and sustainable for long-term use.

The real-world testing and evaluation confirmed that the system operates reliably under practical conditions. The sensor-based counting mechanism produced high accuracy levels and the GPS tracking function remained stable even during continuous movement. The dashboard interfaces proved easy to use and provided valuable insights into bus operations. The administrator and owner panels introduced additional functionality that significantly enhances operational transparency and control.

In conclusion, this project stands as a highly functional, scalable and innovative solution for modernizing public and institutional transport services. It highlights the potential of IoT-driven systems in improving passenger convenience, reducing waiting time, supporting efficient fleet management and contributing toward smarter, more connected urban environments. The developed system offers a strong foundation for future enhancements such as AI-based prediction models, mobile applications and intelligent scheduling systems.

Bibliography

[1] M. Al-Jabi, "Toward an IoT-Enabled Adaptive Interactive Bus Transportation system," *researchgate,* December 2017.

[2] L. R. ,. A. S. ,. J. J. Arun Kumar K, " IoT in Smart Transportation System Intelligent Bus Scheduling System Using IoT: Improving Efficiency, Safety, and Passenger Experience," *researchgate,* March 2025.

[3] A. O. M. A. H. Bhanudas Gadade, "IOT Based Smart School Bus and Student Monitoring System," *NATURALISTA CAMPANO ,* September 2024.

[4] B. A. R. B. V. S. C. Saravanan, "Real-Time Bus Tracking and Management System with IOT, Chatbot Integration and Accident Zone Prediction," *African Journal of Biomedical Research (AJBR),* vol. 28, April 2025.

[5] H. H. ,. M. Y. Dissanayake H.M.V.H, "Design an IoT-based Smart Public Bus Transportation," January 2023.

[6] S. C. V. S. L. S. R. K. Dr D Durga Bhavani, "Implementation of Smart Bus Tracking System Using Wi-Fi," *International Journal of Innovative Research in Science, ,* vol. 6, July 2017.

[7] G. T. S. G V Nivaan, " Smart bus transportation for tracking system: A study case in Indonesia," *Earth and Environmental Science,* 2021.

[8] P. G. S. Keerthi K S, "Smart School Monitoring System Integrated with Smart Bus," *International Journal of Innovative Science and Research Technology,* vol. 10, June 2025.

[9] A. S. M.D.C.J. Gunathialake, "Real-Time GPS-Based Bus Tracking System to Improve Public," *International Research Symposium - IRSSSH-2024,* December 2024.

[10] M. S. M. M. P. K. ,. M. A. M. M. D. Y. M. F. S. ,. M. V. C. Ms. Harshada Patil, "Bus Tracking System," *researchgate,* April 2024.

[11] M. G. N. B. S. K. C. Preeti Acharya, " Final Project Report on Smart Bus Fare NEPAL COLLEGE OF INFORMATION," Lalitpur, Nepal, 2023.

[12] A. S. Süleyman Eken, "A Smart Bus Tracking System Based on Location," *researchgate,* June 2014.

[13] N. F. P. ,. S. ,. M. Jafrul Islam Sojol, "Smart Bus: An Automated Passenger Counting System," *International Journal of Pure and Applied Mathematics,* vol. 118, pp. 3169-3175, January 2018.

[14] Cirkit Design LLC, [Online]. Available: https://app.cirkitdesigner.com/.

[15] draw.io Ltd, "draw.io," draw.io Ltd, Artisans' House, 7 Queensbridge, NN4 7BF, Northampton, England. Company #04051179, 2005. [Online]. Available: https://www.drawio.com/.

[16] [Online]. Available: https://react.dev/.

[17] ailwind Labs Inc, [Online]. Available: https://tailwindcss.com/.

[18] OpenJS Foundation, [Online]. Available: https://nodejs.org/en.

[19] [Online]. Available: https://firebase.google.com/.

A1 Appendix 1 - Code Embed

1 Arduino Code

1.1 Sensor Interrupts (The Core Logic)

This is the most critical part of the code. Instead of constantly checking if a person is walking by (polling), the code uses Interrupt Service Routines (ISRs). When a sensor triggers, the processor pauses whatever it is doing to run this code immediately.

```
void IRAM_ATTR outerSensorISR() {
  if (!isSystemActive) return;
  unsigned long now = millis();
  if (now - lastOuterTime < SENSOR_DEBOUNCE) return;
  lastOuterTime = now;

  if (sequenceState == 2) {
      if (now - sequenceStartTime <= SEQUENCE_TIMEOUT) {
          passengerCount--;
          if (passengerCount < 0) passengerCount = 0;
          dataNeedsSync = true;
      }
      sequenceState = 0;
      updateScreenNeeded = true;
  }
  else if (sequenceState == 0) {
      sequenceState = 1;
      sequenceStartTime = now;
      updateScreenNeeded = true;
  }
}

void IRAM_ATTR innerSensorISR() {
  if (!isSystemActive) return;
  unsigned long now = millis();
  if (now - lastInnerTime < SENSOR_DEBOUNCE) return;
  lastInnerTime = now;
```

```
  if (sequenceState == 1) {
      if (now - sequenceStartTime <= SEQUENCE_TIMEOUT) {
          passengerCount++;
          dataNeedsSync = true;
      }
      sequenceState = 0;
      updateScreenNeeded = true;
  }
  else if (sequenceState == 0) {
      sequenceState = 2;
      sequenceStartTime = now;
      updateScreenNeeded = true;
  }
}
```

- Entry: Outer sensor triggers first (sets sequenceState = 1), followed by the Inner sensor. This adds to the count.
- Exit: Inner sensor triggers first (sets sequenceState = 2), followed by the Outer sensor. This subtracts from the count. The IRAM_ATTR tag tells the ESP32 to keep this code in RAM for maximum speed.

1.2 Network Syncing (Firebase & API)

These functions handle sending data to the cloud. You have two distinct destinations: a custom API and Firebase.

```
void updateFirebaseDataSet(int count) {
  if (WiFi.status() != WL_CONNECTED) return;
  isUploading = true;

  display.clearDisplay();
  display.setCursor(0,20);
  display.setTextSize(1);
  display.println("Uploading Log...");
  display.display();

  String url = firebaseRoot + ".json";
```

```
  HTTPClient http;
  http.begin(url);
  http.addHeader("Content-Type", "application/json");

  String json = "{";
  json += "\"busId\":\"" + busId + "\",";
  json += "\"ownerId\":\"" + ownerId + "\",";
  json += "\"lastUpdated\":" + String((unsigned
long)getEpochMillis()) + ",";
  json += "\"passengers/" + getCurrentTimestampKey() + "\": " +
String(count);
  json += "}";

  http.PATCH(json);
  http.end();

  lastSentCount = count;
  updateScreenNeeded = true;
  isUploading = false;
}
```

- updateFirebaseDataSet: Uses PATCH to update the Firebase Realtime Database. It logs the data under a specific timestamp key.
- syncDataToServer: Uses POST to send current status to your local server (192.168.1.2).

1.3 System Lifecycle (Startup & Shutdown)

- Startup: Turns on the Red LED immediately, waits 2 seconds, then enables WiFi (Green LED) and resets counters.
- Shutdown: Crucially, this runs finalDataUploadBeforeShutdown() to ensure no passenger counts are lost when the user flips the switch, then turns off the LEDs and stops the sensors.

```
void startupSystem() {
  Serial.println("System startup initiated...");

  // 1. Turn on POWER LED (RED) Immediately
  digitalWrite(RED_LED_PIN, HIGH);
  digitalWrite(GREEN_LED_PIN, LOW); // Ensure Green is OFF
initially

  // Show startup message
  display.clearDisplay();
  display.setCursor(0, 20);
  display.setTextSize(1);
  display.println("Starting Up...");
  display.display();

  // 2. WAIT 2 SECONDS (As requested)
  delay(2000);

  // 3. Turn on WIFI LED (GREEN)
  digitalWrite(GREEN_LED_PIN, HIGH);

  // Reset counters for fresh start
  passengerCount = 0;
  lastSentCount = 0;
  sequenceState = 0;
  dataNeedsSync = false;

  // Send online status
  sendStatus("online");

  isSystemActive = true;
  updateScreenNeeded = true;

  Serial.println("System started");
}
void shutdownSystem() {
```

```
  Serial.println("System shutdown initiated...");

  finalDataUploadBeforeShutdown();

  display.clearDisplay();
  display.setCursor(0, 28);
  display.setTextSize(1);
  display.println("  SYSTEM OFF");
  display.display();
  delay(1000);

  digitalWrite(RED_LED_PIN, LOW);
  digitalWrite(GREEN_LED_PIN, LOW);

  display.clearDisplay();
  display.display();

  passengerCount = 0;
  lastSentCount = 0;
  sequenceState = 0;
  dataNeedsSync = false;
  isSystemActive = false;

  Serial.println("System fully shutdown");
}
```

1.4 OLED Display Management

This handles the User Interface (UI), drawing icons and text to the screen.

```
void updateOLED() {
  display.clearDisplay();
```

```
display.fillRect(0, 0, SCREEN_WIDTH, 14, WHITE);
display.setTextColor(BLACK, WHITE);
display.setTextSize(1);
display.setCursor(4, 3);

if (isSystemActive) {
  display.print("Ridelink: ON");
} else {
  display.print("SYSTEM OFF");
}

if (WiFi.status() == WL_CONNECTED) {
   display.drawBitmap(116, 3, wifi_icon, 8, 8, BLACK);
}

display.setTextColor(WHITE);
display.drawBitmap(10, 25, person_icon, 16, 16, WHITE);

display.setCursor(40, 20);
display.setTextSize(3);
display.print(passengerCount);

display.drawLine(0, 48, SCREEN_WIDTH, 48, WHITE);

if (isUploading) {
   display.setCursor(35, 54);
   display.setTextSize(1);
   display.print("Syncing...");
   display.drawBitmap(100, 52, sync_icon, 8, 8, WHITE);
}
else if (sequenceState == 1) {
   display.fillRoundRect(0, 50, 128, 14, 4, WHITE);
   display.setTextColor(BLACK);
   drawCenteredText(">>> ENTERING >>>", 53, 1);
   display.setTextColor(WHITE);
}
```

```
  else if (sequenceState == 2) {
     display.drawRoundRect(0, 50, 128, 14, 4, WHITE);
     drawCenteredText("<<< EXITING <<<", 53, 1);
  }
  else {
     if (hasUnsavedData()) {
       display.setCursor(5, 54);
       display.setTextSize(1);
       display.print("Data Pending");
       display.drawBitmap(110, 54, sync_icon, 8, 8, WHITE);
     } else {
       display.setTextColor(SSD1306_WHITE);
       display.setCursor(0, 54);
       display.setTextSize(1);
       display.print("ID: Bus0001");
     }
  }
  display.display();
}
```

1.5 Full Code

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include "time.h"
#include <sys/time.h>

const char* ssid      = "SLT-ADSL-8E582";
const char* password  = "ub2272098#";
const char* ntpServer = "pool.ntp.org";
const long  gmtOffset_sec = 19800;
const int   daylightOffset_sec = 0;
```

```
const int SWITCH_PIN      = 4;   // Master switch
const int RED_LED_PIN     = 5;   // Online LED
const int GREEN_LED_PIN   = 18;  // WiFi LED
const int PUSHBTN_SEND    = 12;  // Force Send to Firebase
const int PUSHBTN_RESET   = 13;  // Reset Count
// --- SENSOR PINS ---
const int OUTER_PIR_PIN   = 14;  // OUTER Sensor
const int INNER_PIR_PIN   = 27;  // INNER Sensor
String busId = "RidelinkBus0001";
String ownerId = "RidelinkOwner005";
String firebaseRoot = "https://ridelink-26c32-default-
rtdb.firebaseio.com/buses/" + busId;
String passengerApiUrl = "http://192.168.1.2:4002/api/update-
bus";
// OLED Setup
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire,
OLED_RESET);
// Global Variables
bool isSystemActive = false;
bool lastSwitchState = false;
volatile int passengerCount = 0;
volatile int lastSentCount = 0;
volatile bool isUploading = false;
// --- STATE MACHINE ---
volatile int sequenceState = 0;
volatile unsigned long sequenceStartTime = 0;
// TUNING CONSTANTS
const int SEQUENCE_TIMEOUT = 2000;
const int SENSOR_DEBOUNCE = 150;
const unsigned long SEND_INTERVAL = 4000;

volatile unsigned long lastOuterTime = 0;
volatile unsigned long lastInnerTime = 0;
```

```
volatile bool updateScreenNeeded = false;
volatile bool dataNeedsSync = false;
unsigned long lastSendTime = 0;
// INTERRUPT SERVICE ROUTINES
void IRAM_ATTR outerSensorISR() {
  if (!isSystemActive) return;
  unsigned long now = millis();
  if (now - lastOuterTime < SENSOR_DEBOUNCE) return;
  lastOuterTime = now;
  if (sequenceState == 2) {
      if (now - sequenceStartTime <= SEQUENCE_TIMEOUT) {
          passengerCount--;
          if (passengerCount < 0) passengerCount = 0;
          dataNeedsSync = true;
      }
      sequenceState = 0;
      updateScreenNeeded = true;
  }
  else if (sequenceState == 0) {
      sequenceState = 1;
      sequenceStartTime = now;
      updateScreenNeeded = true;
  }
}
void IRAM_ATTR innerSensorISR() {
  if (!isSystemActive) return;
  unsigned long now = millis();
  if (now - lastInnerTime < SENSOR_DEBOUNCE) return;
  lastInnerTime = now;
  if (sequenceState == 1) {
      if (now - sequenceStartTime <= SEQUENCE_TIMEOUT) {
          passengerCount++;
          dataNeedsSync = true;
      }
      sequenceState = 0;
      updateScreenNeeded = true;
```

```
  }
  else if (sequenceState == 0) {
      sequenceState = 2;
      sequenceStartTime = now;
      updateScreenNeeded = true;
  }
}
void IRAM_ATTR resetButtonISR() {
  static unsigned long lastResetTime = 0;
  if (millis() - lastResetTime > 500) {
    passengerCount = 0;
    sequenceState = 0;
    updateScreenNeeded = true;
    dataNeedsSync = true;
    lastResetTime = millis();
  }
}
// Helper Functions
String getCurrentTimestampKey() {
  struct tm timeinfo;
  if(!getLocalTime(&timeinfo)) return "0000-00-
00T00_00_00_000Z";
  char buff[30];
  strftime(buff, sizeof(buff), "%Y-%m-%dT%H_%M_%S", &timeinfo);
  struct timeval tv; gettimeofday(&tv, NULL);
  return String(buff) + "_" + String(tv.tv_usec/1000) + "Z";
}
unsigned long long getEpochMillis() {
  struct timeval tv; gettimeofday(&tv, NULL);
  return ((unsigned long long)tv.tv_sec * 1000) + (tv.tv_usec /
1000);
}
void sendStatus(String status) {
  if (WiFi.status() != WL_CONNECTED) return;
  isUploading = true;
  updateOLED();
```

```
  HTTPClient http;
  http.begin(firebaseRoot + ".json");
  http.addHeader("Content-Type", "application/json");
  String json = "{\"status\":\"" + status +
"\",\"lastUpdated\":" + String((unsigned long)getEpochMillis())
+ "}";
  http.PATCH(json);
  http.end();

  isUploading = false;
  updateScreenNeeded = true;
}
void syncDataToServer() {
  if (WiFi.status() != WL_CONNECTED) return;
  isUploading = true;
  updateOLED();
  HTTPClient http;
  http.begin(passengerApiUrl);
  http.addHeader("Content-Type", "application/json");
  String json = "{\"busId\":\"" + busId +
"\",\"lat\":6.9271,\"lon\":79.8612,\"status\":\"online\",\"pass
engers\":" + String(passengerCount) + "}";
  int code = http.POST(json);
  if (code > 0) { dataNeedsSync = false; }
  http.end();
  isUploading = false;
  updateScreenNeeded = true;
}
void updateFirebaseDataSet(int count) {
  if (WiFi.status() != WL_CONNECTED) return;
  isUploading = true;
  display.clearDisplay();
  display.setCursor(0,20);
  display.setTextSize(1);
  display.println("Uploading Log...");
  display.display();
```

```
  String url = firebaseRoot + ".json";
  HTTPClient http;
  http.begin(url);
  http.addHeader("Content-Type", "application/json");
  String json = "{";
  json += "\"busId\":\"" + busId + "\",";
  json += "\"ownerId\":\"" + ownerId + "\",";
  json += "\"lastUpdated\":" + String((unsigned
long)getEpochMillis()) + ",";
  json += "\"passengers/" + getCurrentTimestampKey() + "\": " +
String(count);
  json += "}";
  http.PATCH(json);
  http.end();
  lastSentCount = count;
  updateScreenNeeded = true;
  isUploading = false;
}
bool hasUnsavedData() {
  return (passengerCount != lastSentCount) || dataNeedsSync;
}
void finalDataUploadBeforeShutdown() {
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("WiFi disconnected. Cannot upload data.");
    return;
  }
  display.clearDisplay();
  display.setCursor(0, 20);
  display.setTextSize(1);
  display.println("Shutting Down...");
  display.println("Syncing Status...");
  display.display();
  if (hasUnsavedData()) {
      Serial.println("Uploading unsaved data...");
      updateFirebaseDataSet(passengerCount);
      HTTPClient http;
```

```
      http.begin(passengerApiUrl);
      http.addHeader("Content-Type", "application/json");
      String json = "{\"busId\":\"" + busId +
"\",\"lat\":6.9271,\"lon\":79.8612,\"status\":\"offline\",\"pas
sengers\":" + String(passengerCount) + "}";
      http.POST(json);
      http.end();
  }
  Serial.println("Sending OFFLINE status...");
  HTTPClient http2;
  http2.begin(firebaseRoot + ".json");
  http2.addHeader("Content-Type", "application/json");
  String statusJson =
"{\"status\":\"offline\",\"lastUpdated\":" + String((unsigned
long)getEpochMillis()) + "}";
  http2.PATCH(statusJson);
  http2.end();
  dataNeedsSync = false;
}
// SHUTDOWN SYSTEM
void shutdownSystem() {
  Serial.println("System shutdown initiated...");
  finalDataUploadBeforeShutdown();
  display.clearDisplay();
  display.setCursor(0, 28);
  display.setTextSize(1);
  display.println("  SYSTEM OFF");
  display.display();
  delay(1000);
  digitalWrite(RED_LED_PIN, LOW);
  digitalWrite(GREEN_LED_PIN, LOW);
  display.clearDisplay();
  display.display();
  passengerCount = 0;
  lastSentCount = 0;
  sequenceState = 0;
```

```
  dataNeedsSync = false;
  isSystemActive = false;
  Serial.println("System fully shutdown");
}
// STARTUP SYSTEM (MODIFIED)
void startupSystem() {
  Serial.println("System startup initiated...");
  // 1. Turn on POWER LED (RED) Immediately
  digitalWrite(RED_LED_PIN, HIGH);
  digitalWrite(GREEN_LED_PIN, LOW); // Ensure Green is OFF
initially
  // Show startup message
  display.clearDisplay();
  display.setCursor(0, 20);
  display.setTextSize(1);
  display.println("Starting Up...");
  display.display();
  // 2. WAIT 2 SECONDS (As requested)
  delay(2000);
  // 3. Turn on WIFI LED (GREEN)
  digitalWrite(GREEN_LED_PIN, HIGH);
  // Reset counters for fresh start
  passengerCount = 0;
  lastSentCount = 0;
  sequenceState = 0;
  dataNeedsSync = false;
  // Send online status
  sendStatus("online");
  isSystemActive = true;
  updateScreenNeeded = true;
  Serial.println("System started");
}


// UI ASSETS
```

```
const unsigned char PROGMEM wifi_icon[] = { 0x3C, 0x42, 0x81,
0x3C, 0x42, 0x18, 0x00, 0x18 };
const unsigned char PROGMEM sync_icon[] = { 0x00, 0x38, 0x7C,
0x5E, 0x1F, 0x1F, 0x0E, 0x00 };
const unsigned char PROGMEM person_icon[] = {
  0x00, 0x00, 0x03, 0xC0, 0x07, 0xE0, 0x07, 0xE0,
  0x03, 0xC0, 0x00, 0x00, 0x0F, 0xF0, 0x1F, 0xF8,
  0x1C, 0x38, 0x38, 0x1C, 0x30, 0x0C, 0x30, 0x0C,
  0x30, 0x0C, 0x30, 0x0C, 0x30, 0x0C, 0x00, 0x00
};
void drawCenteredText(String text, int y, int size) {
  int16_t x1, y1;
  uint16_t w, h;
  display.setTextSize(size);
  display.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
  int x = (SCREEN_WIDTH - w) / 2;
  display.setCursor(x, y);
  display.print(text);
}
void updateOLED() {
  display.clearDisplay();
  display.fillRect(0, 0, SCREEN_WIDTH, 14, WHITE);
  display.setTextColor(BLACK, WHITE);
  display.setTextSize(1);
  display.setCursor(4, 3);
  if (isSystemActive) {
    display.print("Ridelink: ON");
  } else {
    display.print("SYSTEM OFF");
  }
  if (WiFi.status() == WL_CONNECTED) {
     display.drawBitmap(116, 3, wifi_icon, 8, 8, BLACK);
  }
  display.setTextColor(WHITE);
  display.drawBitmap(10, 25, person_icon, 16, 16, WHITE);
  display.setCursor(40, 20);
```

```
  display.setTextSize(3);
  display.print(passengerCount);
  display.drawLine(0, 48, SCREEN_WIDTH, 48, WHITE);
  if (isUploading) {
     display.setCursor(35, 54);
     display.setTextSize(1);
     display.print("Syncing...");
     display.drawBitmap(100, 52, sync_icon, 8, 8, WHITE);
  }
  else if (sequenceState == 1) {
     display.fillRoundRect(0, 50, 128, 14, 4, WHITE);
     display.setTextColor(BLACK);
     drawCenteredText(">>> ENTERING >>>", 53, 1);
     display.setTextColor(WHITE);
  }
  else if (sequenceState == 2) {
     display.drawRoundRect(0, 50, 128, 14, 4, WHITE);
     drawCenteredText("<<< EXITING <<<", 53, 1);
  }
  else {
     if (hasUnsavedData()) {
       display.setCursor(5, 54);
       display.setTextSize(1);
       display.print("Data Pending");
       display.drawBitmap(110, 54, sync_icon, 8, 8, WHITE);
     } else {
       display.setTextColor(SSD1306_WHITE);
       display.setCursor(0, 54);
       display.setTextSize(1);
       display.print("ID: Bus0001");
     }
  }
  display.display();
}

// SETUP (MODIFIED)
```

```
void setup() {
  Serial.begin(115200);
  pinMode(SWITCH_PIN, INPUT_PULLUP);
  // LED SETUP: RED ON Immediately
  pinMode(RED_LED_PIN, OUTPUT);
  digitalWrite(RED_LED_PIN, HIGH); // POWER LED ON
  pinMode(GREEN_LED_PIN, OUTPUT);
  digitalWrite(GREEN_LED_PIN, LOW); // WIFI LED OFF initially
  pinMode(PUSHBTN_SEND, INPUT_PULLUP);
  // SENSOR PINS
  pinMode(OUTER_PIR_PIN, INPUT_PULLDOWN);
  attachInterrupt(digitalPinToInterrupt(OUTER_PIR_PIN),
outerSensorISR, RISING);
  pinMode(INNER_PIR_PIN, INPUT_PULLDOWN);
  attachInterrupt(digitalPinToInterrupt(INNER_PIR_PIN),
innerSensorISR, RISING);
  pinMode(PUSHBTN_RESET, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(PUSHBTN_RESET),
resetButtonISR, FALLING);
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("OLED FAILED");
    while(1);
  }
  display.clearDisplay();
  display.setTextColor(WHITE);
  display.setTextSize(1);
  display.println("Connecting WiFi...");
  display.display();
  WiFi.setSleep(false);

  WiFi.begin(ssid, password);
  int retry = 0;
  while (WiFi.status() != WL_CONNECTED && retry < 20) {
    delay(300);
    Serial.print(".");
    retry++;
```

```
  }

  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\nWiFi Connected");
    // Ensure Green LED is ON if connected (Setup phase)
    digitalWrite(GREEN_LED_PIN, HIGH);
  } else {
    Serial.println("\nWiFi Connection Failed");
  }

  configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);

  // Initialize switch state
  lastSwitchState = (digitalRead(SWITCH_PIN) == LOW);
  lastSentCount = 0;
  updateScreenNeeded = true;
}
// Main Loop
void loop() {
  unsigned long currentMillis = millis();

  bool currentSwitchState = (digitalRead(SWITCH_PIN) == LOW);

  if (currentSwitchState != lastSwitchState) {
    lastSwitchState = currentSwitchState;
    if (currentSwitchState) {
      startupSystem();
    } else {
      shutdownSystem();
      return;
    }
  }

  if (!isSystemActive) {
    delay(100);
    return;
```

```
  }

  if (updateScreenNeeded) {
    updateOLED();
    updateScreenNeeded = false;
  }

  if (sequenceState != 0) {
    if (currentMillis - sequenceStartTime > SEQUENCE_TIMEOUT) {
      sequenceState = 0;
      updateScreenNeeded = true;
    }
  }

  if (dataNeedsSync) {
     if (currentMillis - lastSendTime > SEND_INTERVAL) {
        syncDataToServer();
        lastSendTime = currentMillis;
     }
  }

  if (digitalRead(PUSHBTN_SEND) == LOW) {
     delay(50);
     if (digitalRead(PUSHBTN_SEND) == LOW) {
        Serial.println("Manual send button pressed");
        updateFirebaseDataSet(passengerCount);
        while(digitalRead(PUSHBTN_SEND) == LOW) delay(10);
     }
  }
}
```

2 Backend

2.1 Firebase Initialization

This part initializes Firebase using your project's configuration and sets up a reference to the real-time database. firebaseApp connects your Node.js server to Firebase, and db allows you to read and write data from the Realtime Database.

```
const firebaseApp = initializeApp(firebaseConfig);
const db = getDatabase(firebaseApp);
```

2.2 Express App & Socket.IO Setup

This sets up an Express server for handling HTTP requests and integrates Socket.IO for real-time communication. cors() enables cross-origin requests, while express.json() allows parsing JSON request bodies. The io object allows broadcasting bus updates to connected clients in real time.

```
const app = express();
const server = http.createServer(app);
const io = new Server(server, { cors: { origin: "*" } });

app.use(cors());
app.use(express.json());
```

2.3 Update Bus Location & Passenger Count

This endpoint updates a bus's current location, number of passengers, and status. It stores the information in a local busData object, broadcasts the update to all connected clients using Socket.IO, and also updates the Firebase Realtime Database with the latest info.

```
app.post("/api/update-bus", async (req, res) => {
  try {
    const { busId, lat, lon, passengers, status } = req.body;

    if (!busId || typeof lat !== "number" || typeof lon !== "number")
{
      return res.status(400).json({ error: "Missing or invalid busId,
lat, or lon" });
    }

    const passengerCount =
      passengers !== undefined && passengers !== null ?
Number(passengers) : 0;

    busData[busId] = {
      lat,
```

```
      lon,
      passengers: passengerCount,
      status: status,
      lastUpdate: new Date().toISOString()
    };

    io.emit("busLocationUpdate", {
      busId,
      lat,
      lon,
      passengers: passengerCount,
      status: status
    });

    const busRef = ref(db, `buses/${busId}`);
    const updatePayload = {
      lat,
      lon,
      status: status,
      lastUpdated: Date.now()
    };

    await update(busRef, updatePayload);

    return res.json({
      success: true,
      message: `Bus ${busId} location & passengers updated
successfully`
    });
  } catch (error) {
    return res.status(500).json({ error: "Internal Server Error" });
  }
});
```

2.4 Update Bus Status Only

This endpoint specifically updates only the bus's status (online/offline). It updates the local cache busData, emits the change to all clients via Socket.IO, and syncs the status to Firebase without changing other bus information like coordinates.

```
app.post("/api/update-bus-status", async (req, res) => {
  try {
    const { busId, status } = req.body;

    if (!busId || (status !== "online" && status !== "offline")) {
```

```
      return res.status(400).json({ error: "Missing or invalid busId
or status" });
    }

    if (busData[busId]) {
      busData[busId].status = status;
      busData[busId].lastUpdate = new Date().toISOString();
    } else {
      busData[busId] = {
        status,
        lastUpdate: new Date().toISOString(),
        lat: null,
        lon: null,
        passengers: 0
      };
    }

    io.emit("busStatusUpdate", { busId, status });

    const busRef = ref(db, `buses/${busId}`);
    await update(busRef, {
      status,
      lastUpdated: Date.now()
    });

    return res.json({
      success: true,
      message: `Bus ${busId} status updated to ${status} successfully`
    });
  } catch (error) {
    return res.status(500).json({ error: "Internal Server Error" });
  }
});
```

2.5 Get All Bus Data

This endpoint returns the current state of all buses stored in the server's busData object. It allows clients to retrieve the latest bus locations, passenger counts, and statuses without needing real-time updates.

```
app.get("/api/buses", (req, res) => res.json(busData));
```

2.6 Full Code

```
import cors from "cors";
import express from "express";
import { initializeApp } from "firebase/app";
import { getDatabase, ref, update } from "firebase/database";
import http from "http";
import { Server } from "socket.io";

const firebaseConfig = {
  apiKey: "AIzaSyARWCfQFeHb-4OKjJIoeP6oeMajx1nHhkE",
  authDomain: "ridelink-26c32.firebaseapp.com",
  databaseURL: "https://ridelink-26c32-default-
rtdb.firebaseio.com",
  projectId: "ridelink-26c32",
  storageBucket: "ridelink-26c32.appspot.com",
  messagingSenderId: "296583693513",
  appId: "1:296583693513:web:3dcf1a741cab1dc22af918",
  measurementId: "G-5STSMT9RWJ"
};

const firebaseApp = initializeApp(firebaseConfig);
const db = getDatabase(firebaseApp);

const app = express();
const server = http.createServer(app);
const io = new Server(server, { cors: { origin: "*" } });

app.use(cors());
app.use(express.json());

let busData = {};

app.post("/api/update-bus", async (req, res) => {
  try {
    const { busId, lat, lon, passengers, status } = req.body;

    if (!busId || typeof lat !== "number" || typeof lon !==
"number") {
      return res.status(400).json({ error: "Missing or invalid
busId, lat, or lon" });
    }

    const passengerCount =
      passengers !== undefined && passengers !== null ?
Number(passengers) : 0;
```

```
    busData[busId] = {
      lat,
      lon,
      passengers: passengerCount,
      status: status,
      lastUpdate: new Date().toISOString()
    };

    io.emit("busLocationUpdate", {
      busId,
      lat,
      lon,
      passengers: passengerCount,
      status: status
    });

    const busRef = ref(db, `buses/${busId}`);
    const updatePayload = {
      lat,
      lon,
      status: status,
      lastUpdated: Date.now()
    };

    await update(busRef, updatePayload);

    return res.json({
      success: true,
      message: `Bus ${busId} location & passengers updated
successfully`
    });
  } catch (error) {
    return res.status(500).json({ error: "Internal Server Error" });
  }
});

app.post("/api/update-bus-status", async (req, res) => {
  try {
    const { busId, status } = req.body;

    if (!busId || (status !== "online" && status !== "offline")) {
      return res.status(400).json({ error: "Missing or invalid busId
or status" });
    }
```

```
    if (busData[busId]) {
      busData[busId].status = status;
      busData[busId].lastUpdate = new Date().toISOString();
    } else {
      busData[busId] = {
        status,
        lastUpdate: new Date().toISOString(),
        lat: null,
        lon: null,
        passengers: 0
      };
    }

    io.emit("busStatusUpdate", { busId, status });

    const busRef = ref(db, `buses/${busId}`);
    await update(busRef, {
      status,
      lastUpdated: Date.now()
    });

    return res.json({
      success: true,
      message: `Bus ${busId} status updated to ${status} successfully`
    });
  } catch (error) {
    return res.status(500).json({ error: "Internal Server Error" });
  }
});

app.get("/api/buses", (req, res) => res.json(busData));

io.on("connection", (socket) => {
  socket.emit("initialData", busData);
});

server.listen(4002, () => console.log("Live API running on
http://localhost:4002"));
```

A2 Appendix 2 - Landscape Pages

1 UI Designs

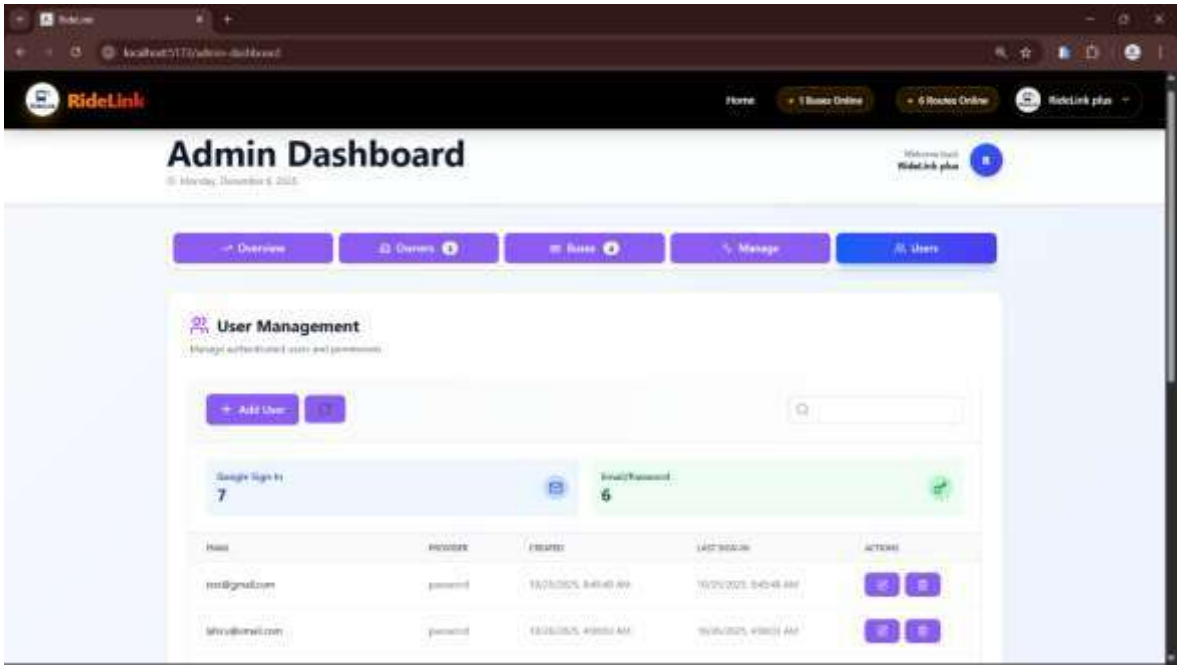

Figure 4.1 Admin Dashboard Interface

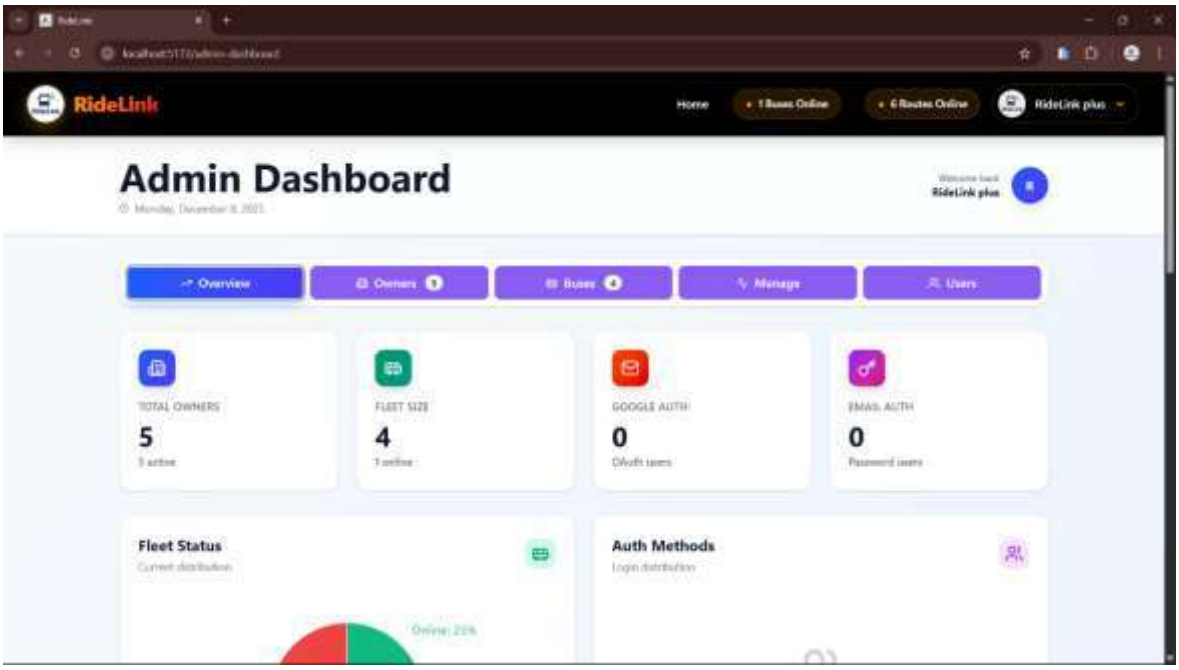

Figure 4.2 Admin Dashboard Interface

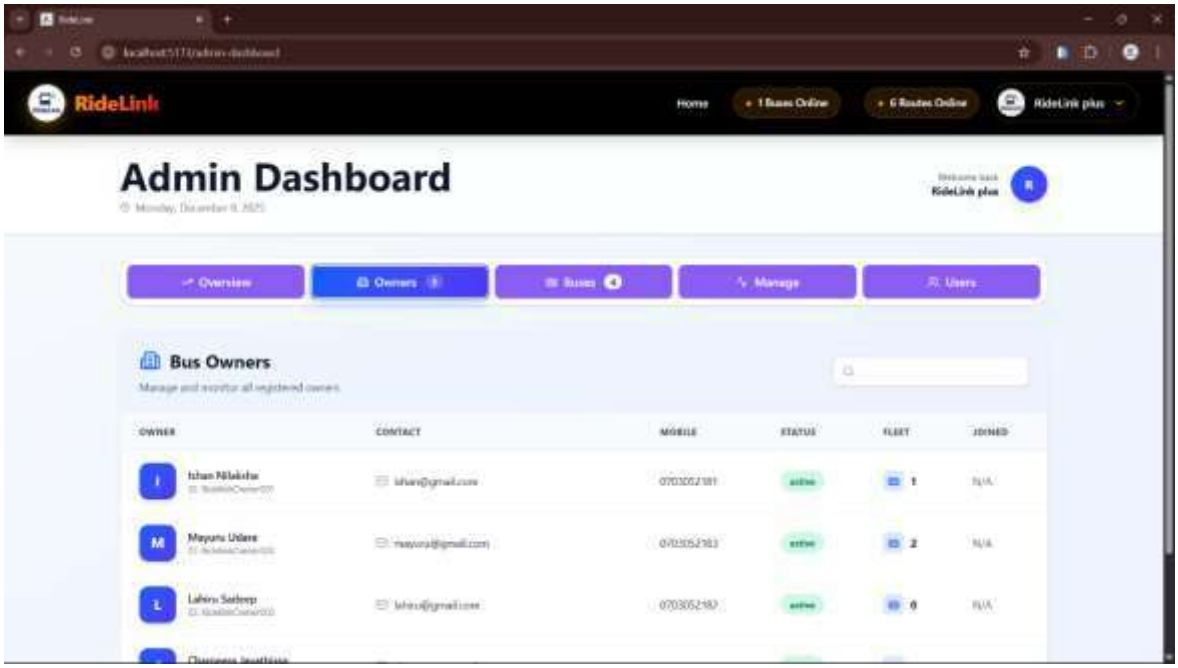

Figure 4.3 Admin Dashboard Interface

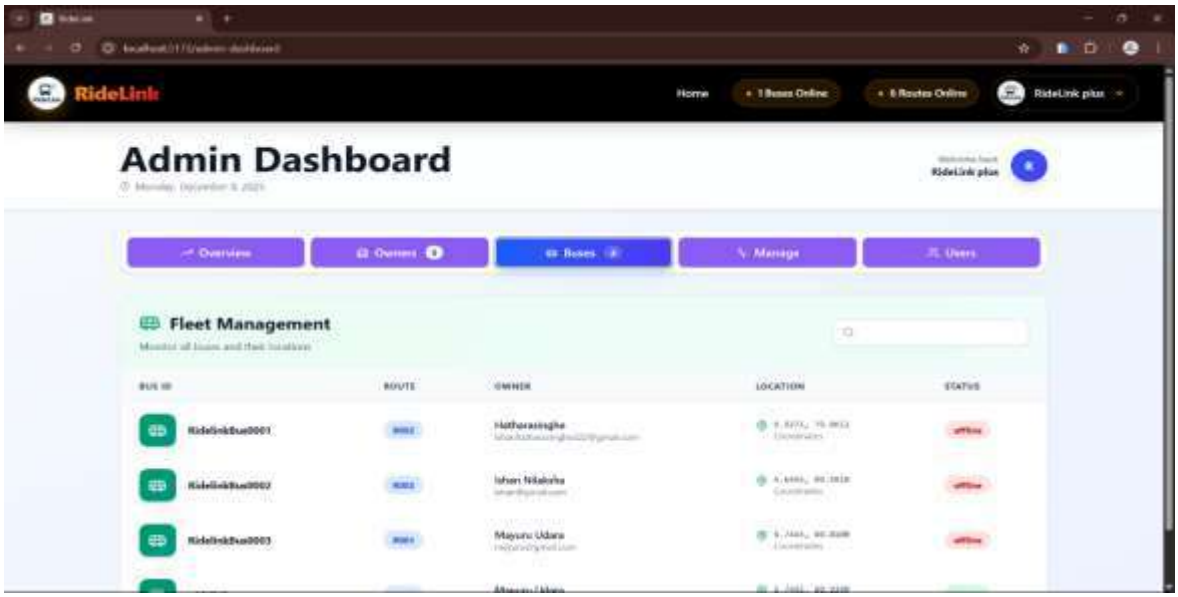

Figure 4.4 Admin Dashboard Interface

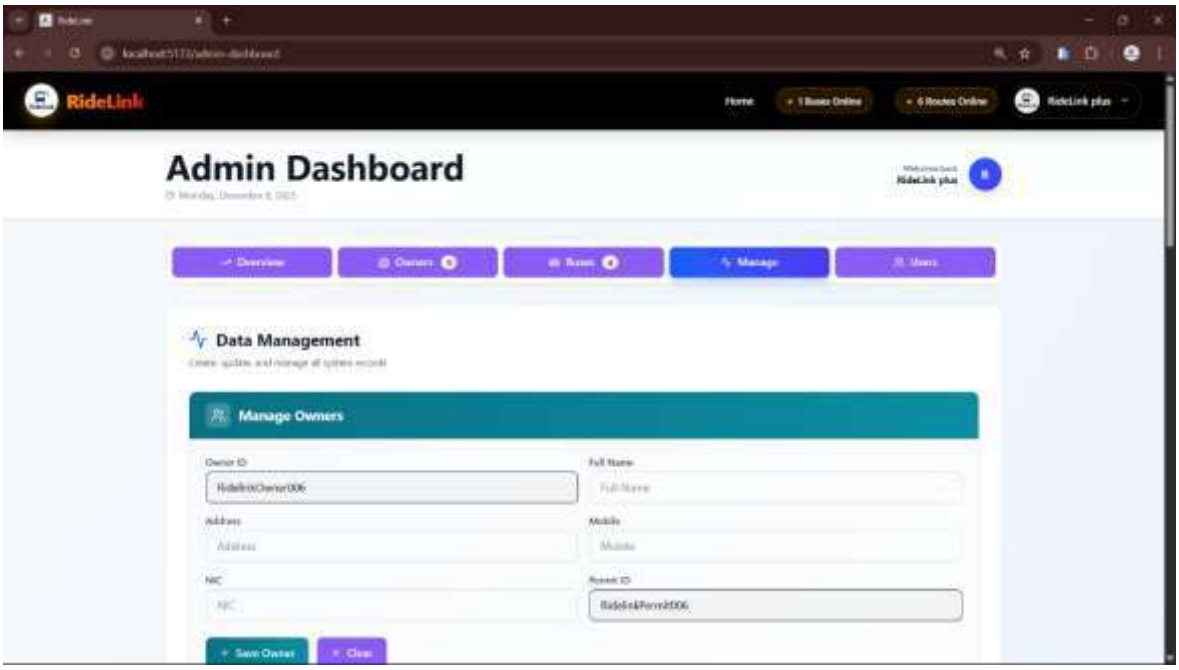

Figure 4.5 Admin Dashboard Interface

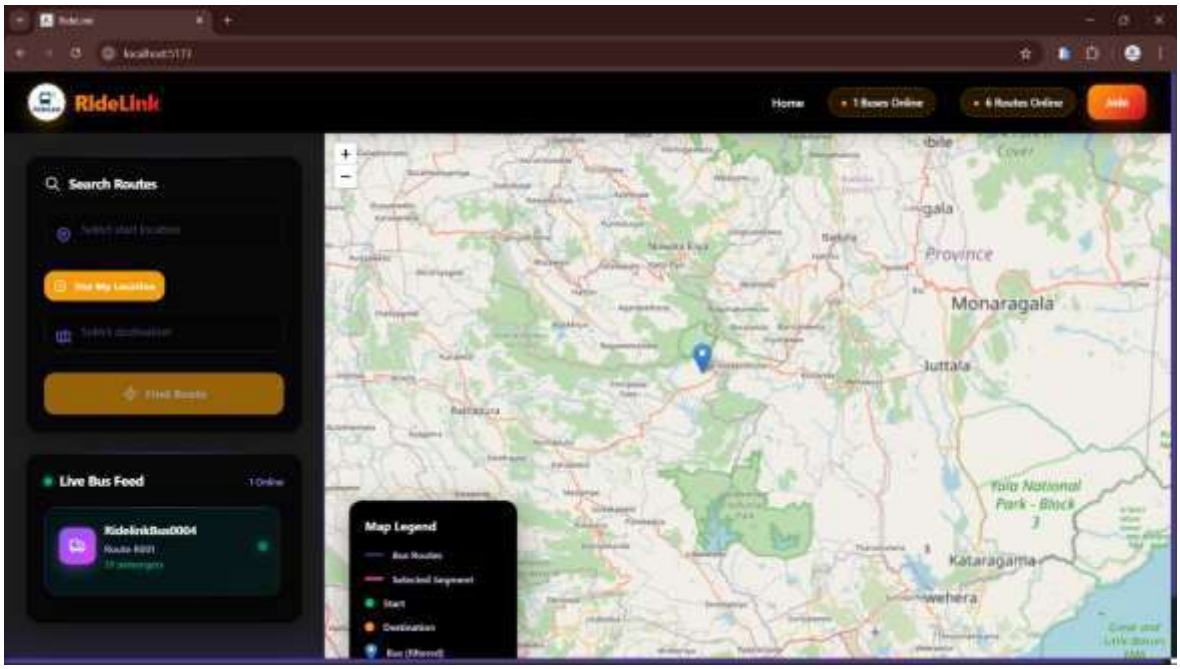

Figure 4.6 Full view of Main Screen

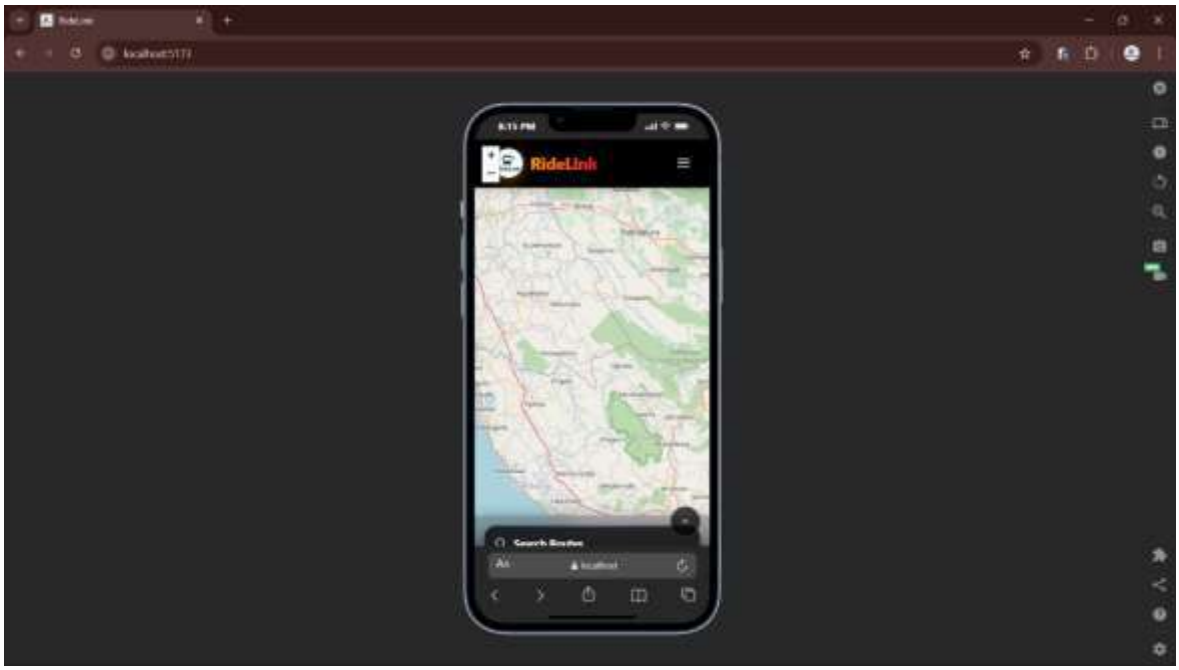

Figure 4.7 Mobile view of Main Screen

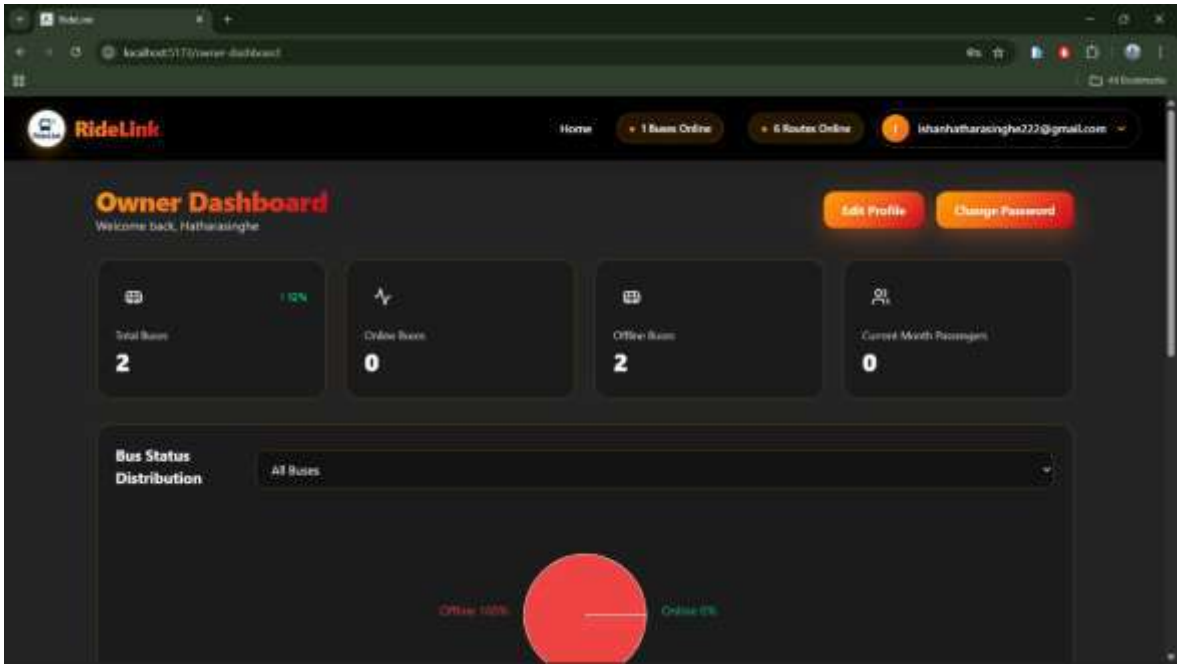

Figure 4.8 Full view of Owner Dashboard Interface

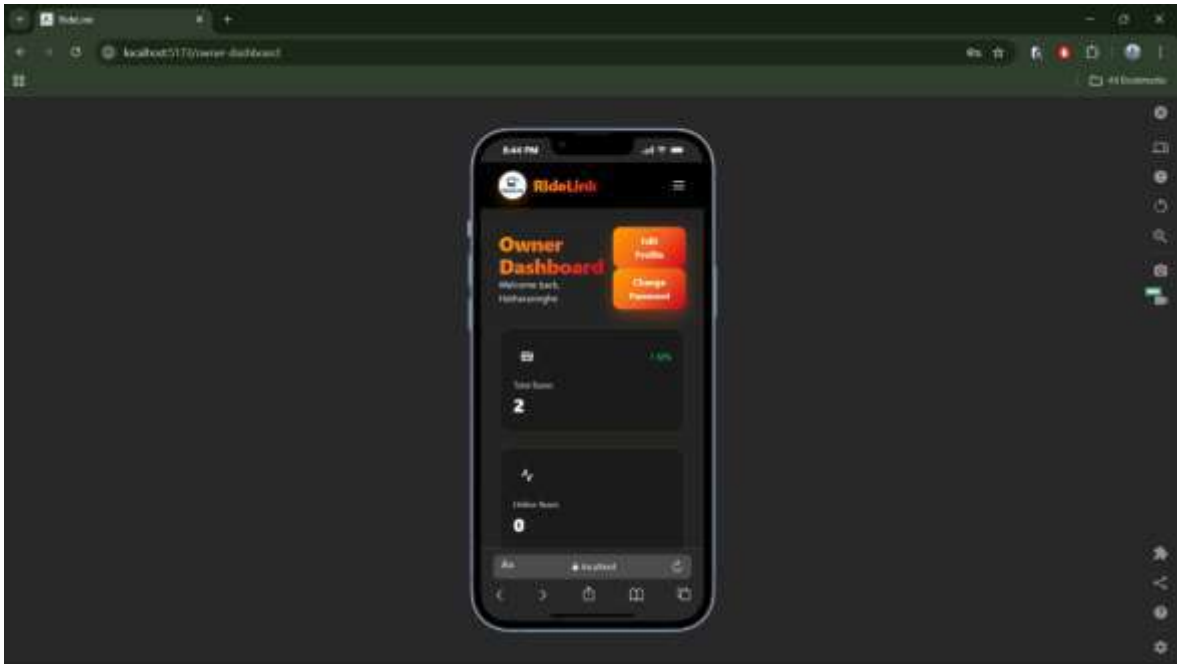

Figure 4.9 Mobile view of Owner Dashboard Interface

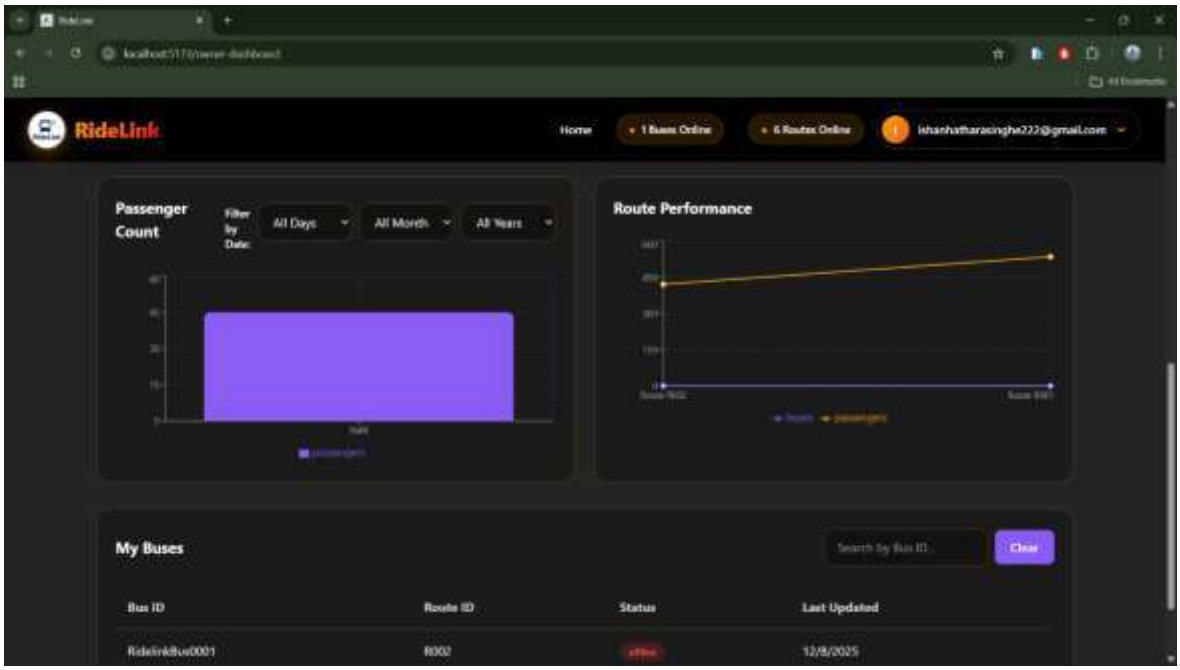

Figure 4.10 Full view of Owner Dashboard Interface

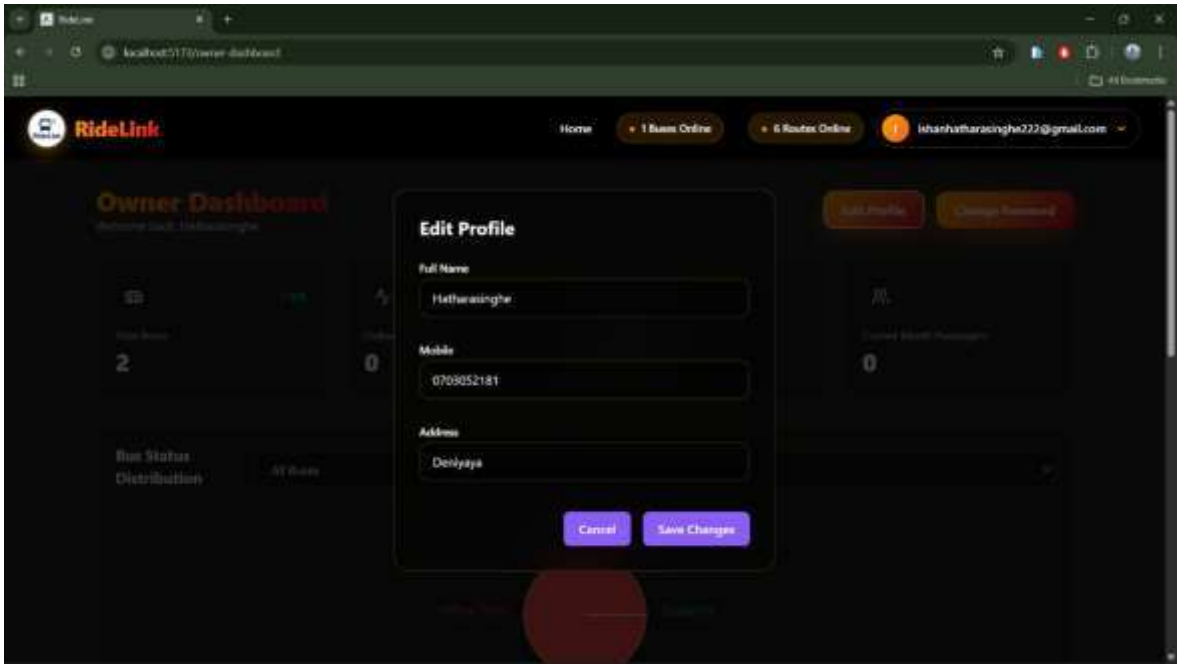

Figure 4.11 Full view of Owner Dashboard Interface

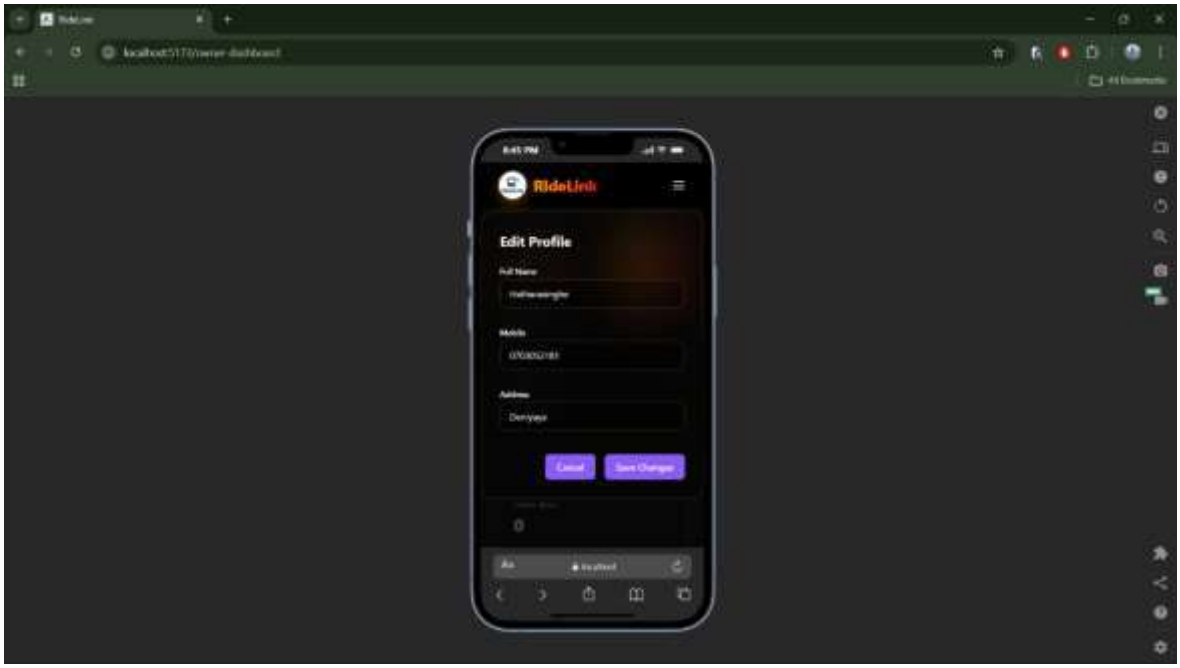

Figure 4.12 Mobile view of Owner Dashboard Interface

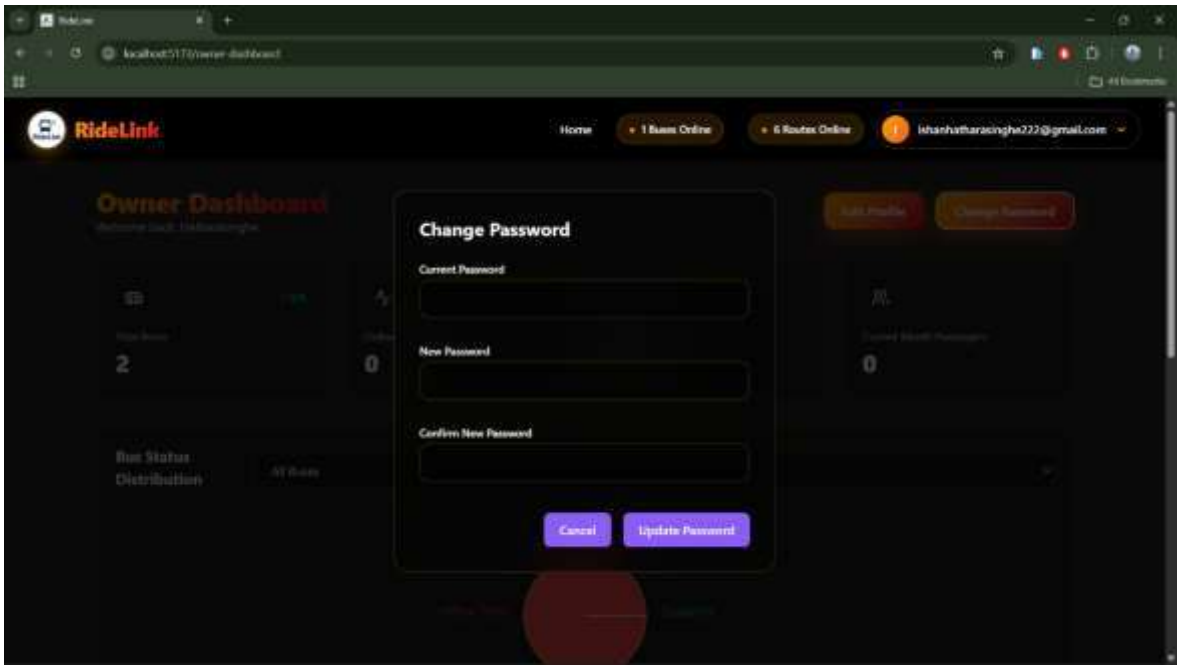

Figure 4.13 Full view of Owner Dashboard Interface

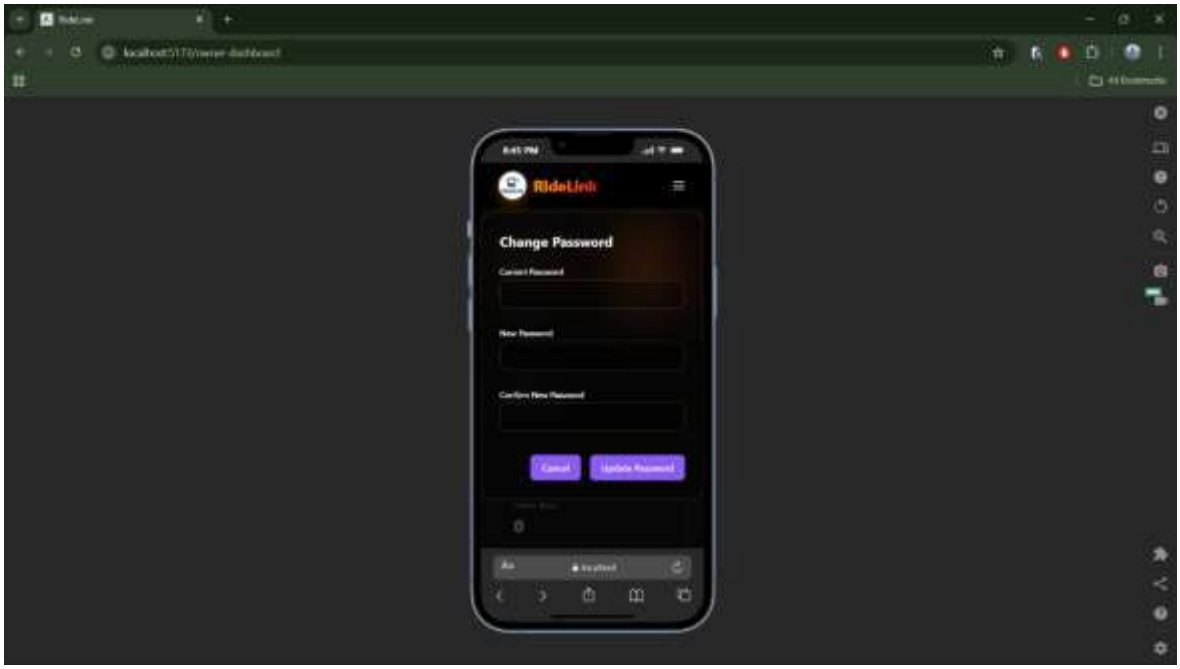

Figure 4.14 Mobile view of Owner Dashboard Interface

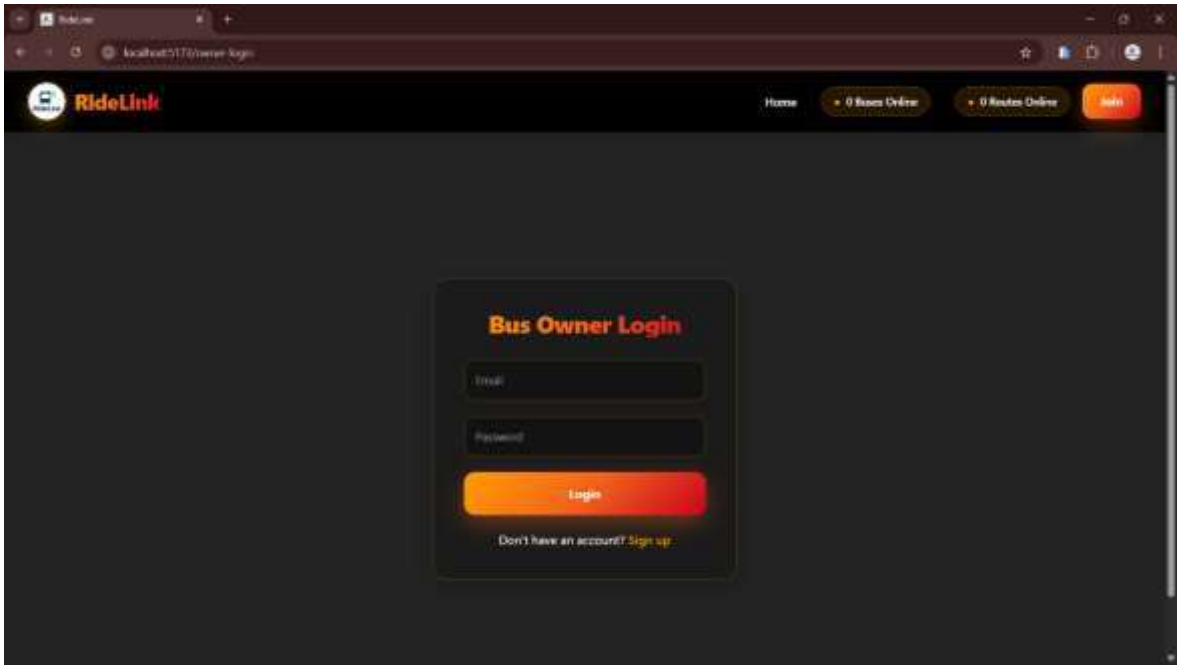

Figure 4.15 Full view of Owner Login Interface

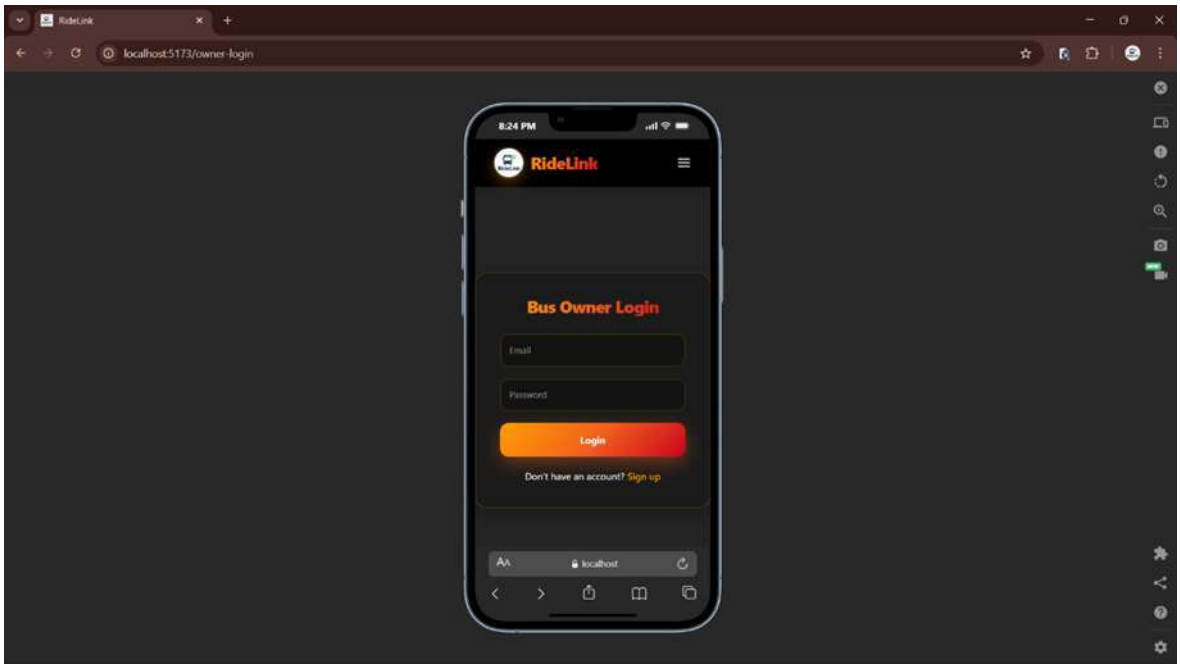

Figure 4.16 Mobile view of Owner Login Interface

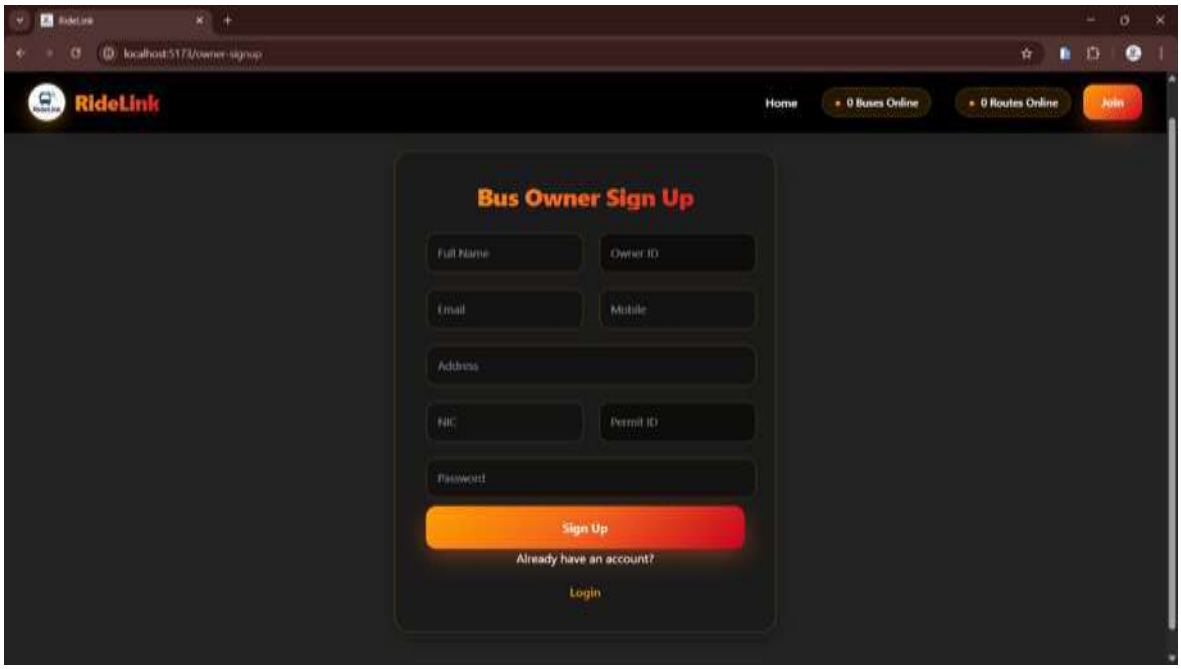

Figure 4.17 Full view of Owner Sign Up Interface

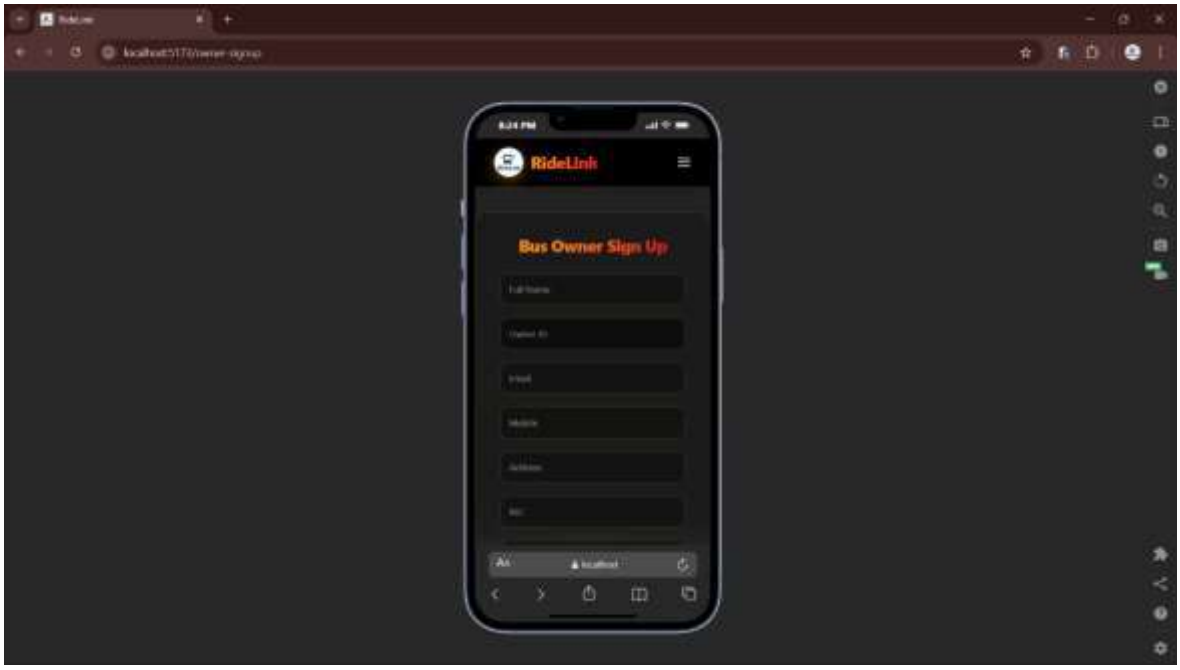

Figure 4.18 Mobile view of Owner Sign Up Interface

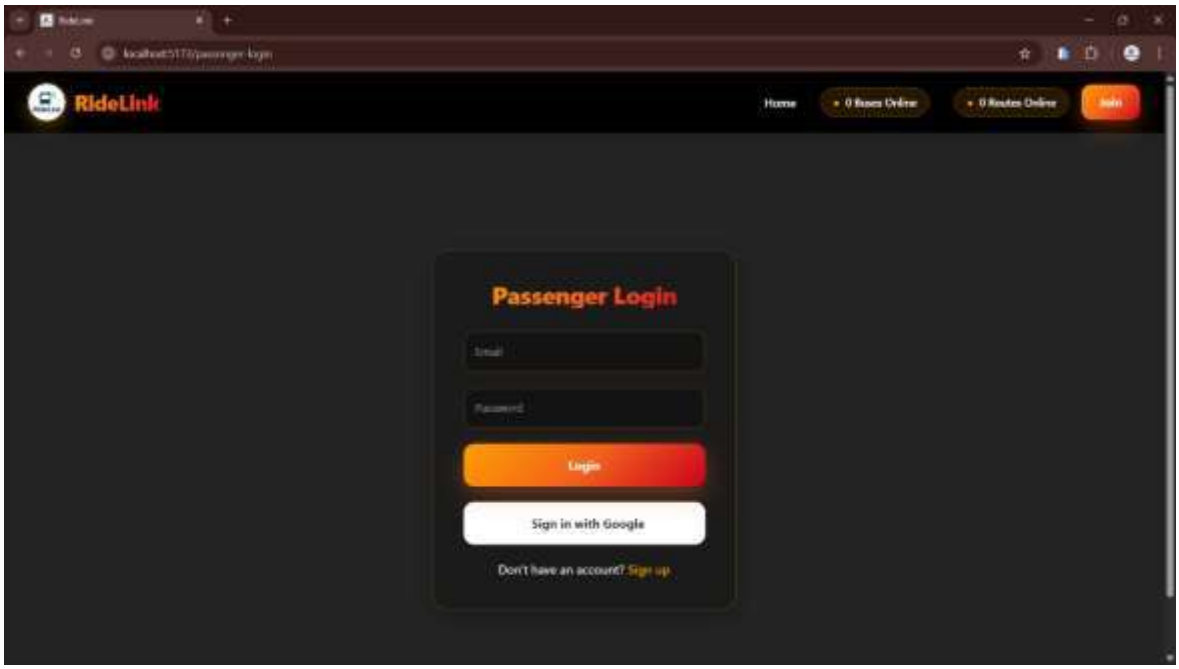

Figure 4.19 Full view of Passenger Login Interface

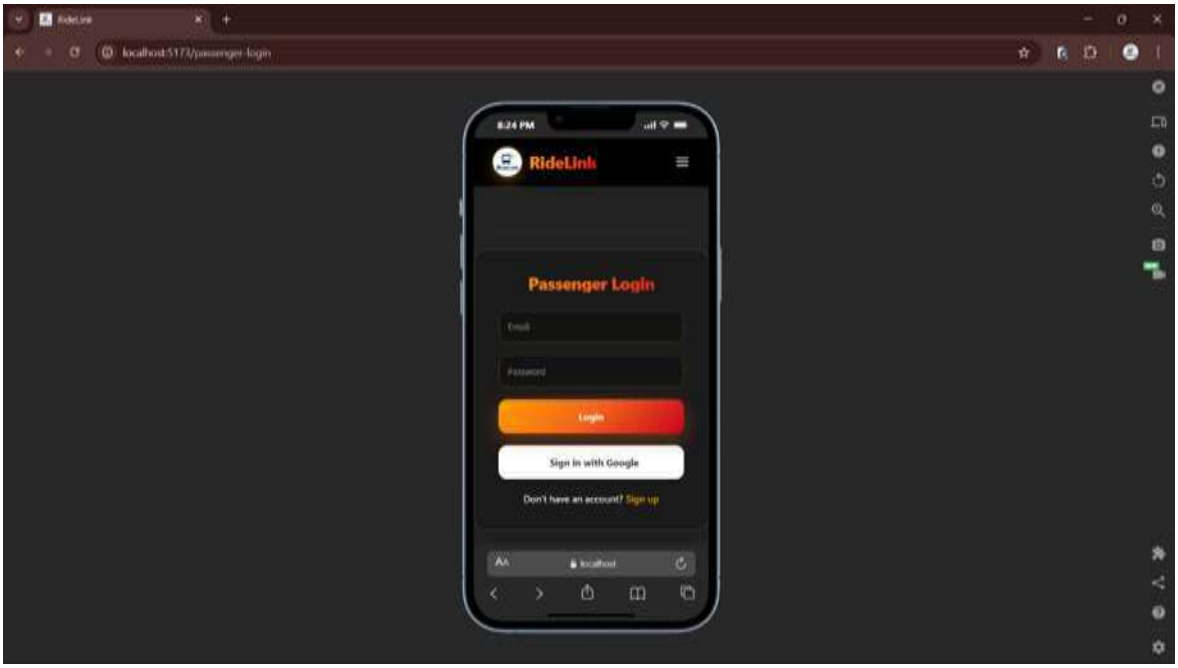

Figure 4.20 Mobile view of Passenger Login Interface

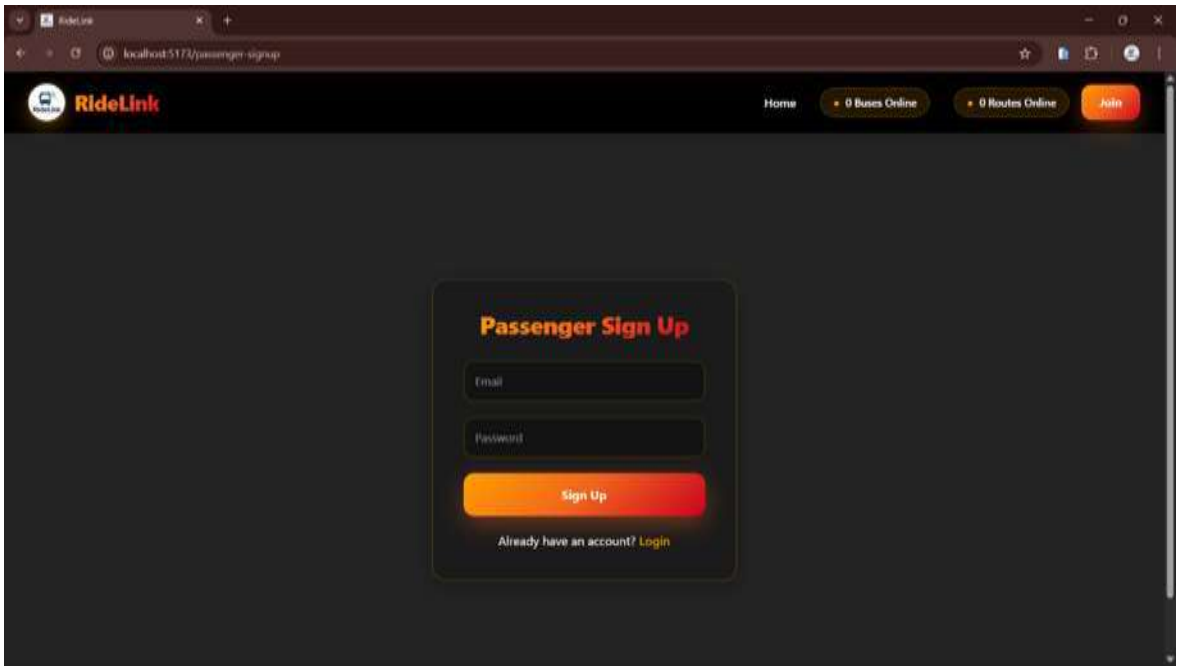

Figure 4.21 Full view of Passenger Sign Up Interface

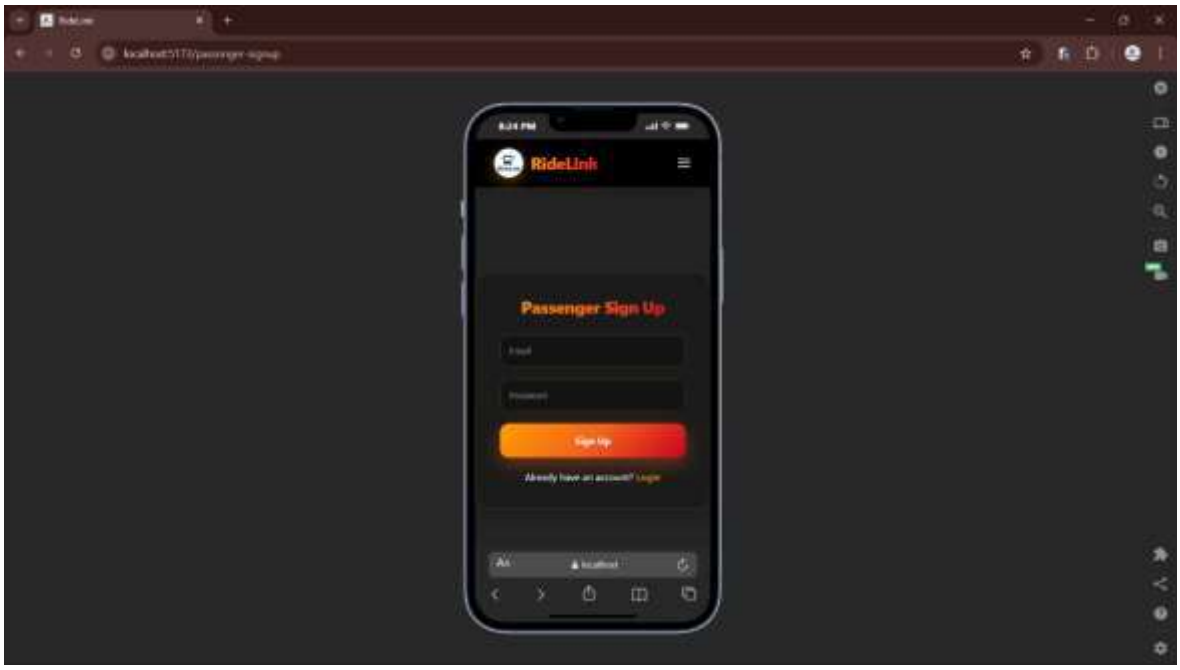

Figure 4.22 Mobile view of Passenger Sign Up Interface

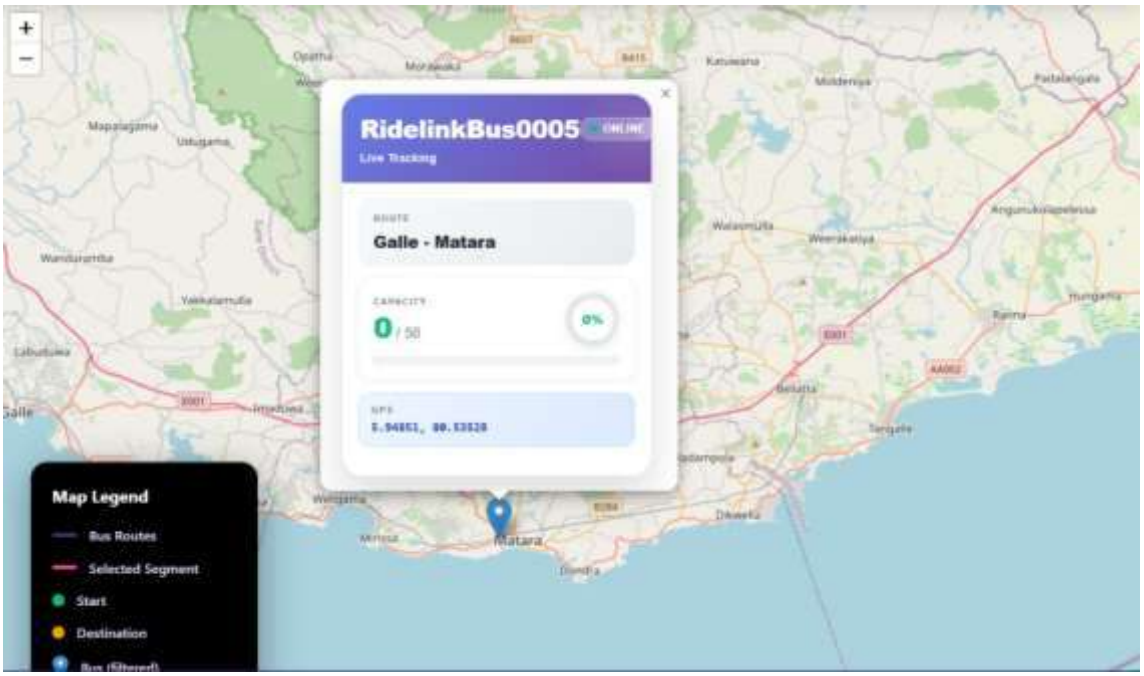

Figure 4.23 Full view of Status Indicator Interface

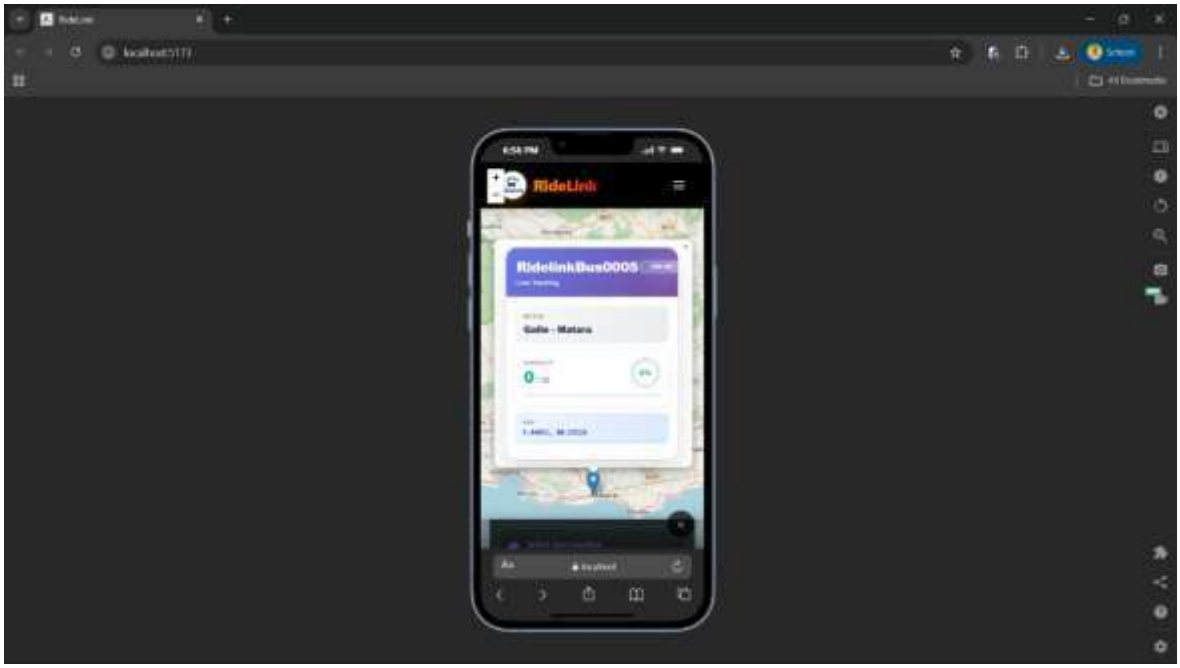

Figure 4.24 Mobile view of Status Indicator Interface

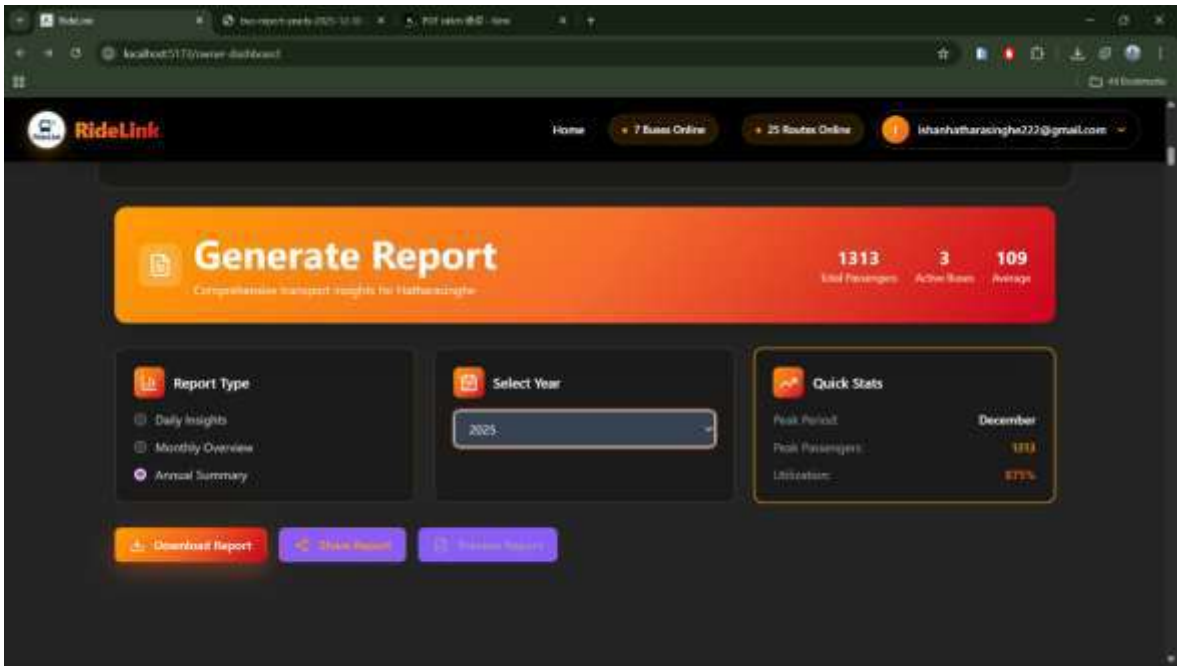

Figure 4.25 Report Generation Interface

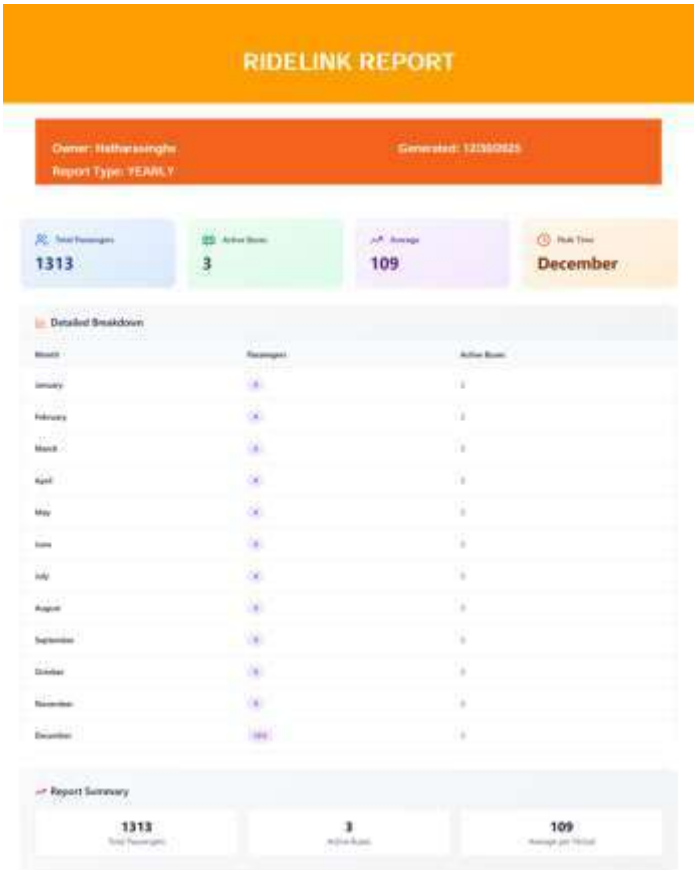

Figure 4.26 Report

2 UI Codes

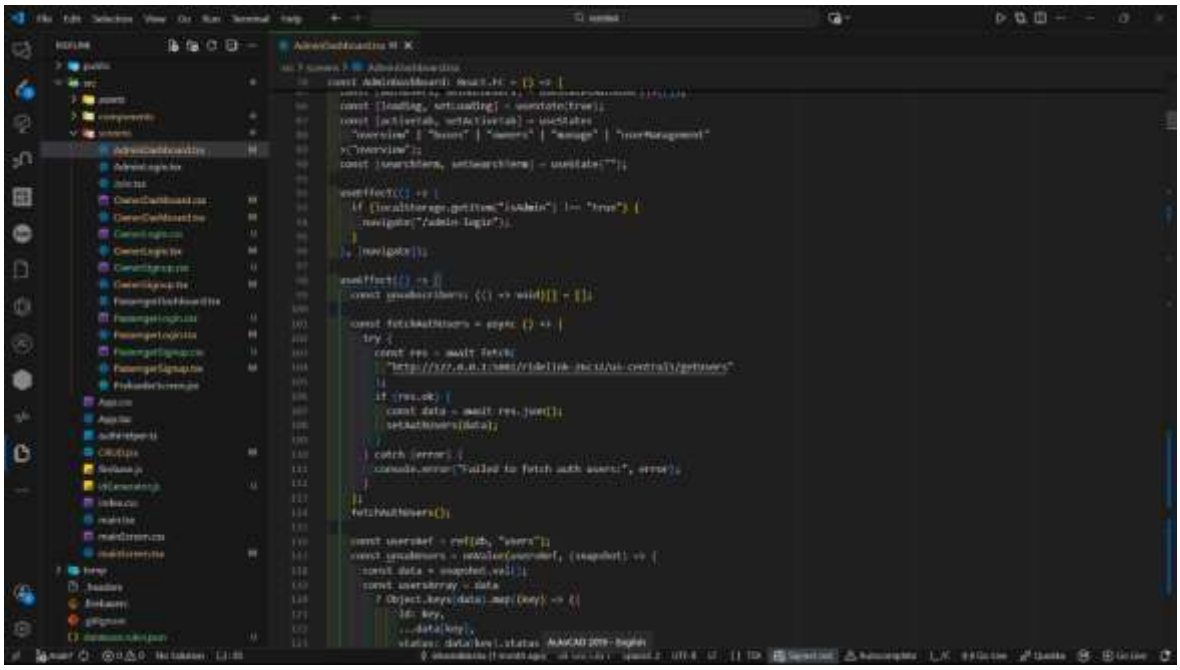

Figure 4.27 Admin Dashboard Code

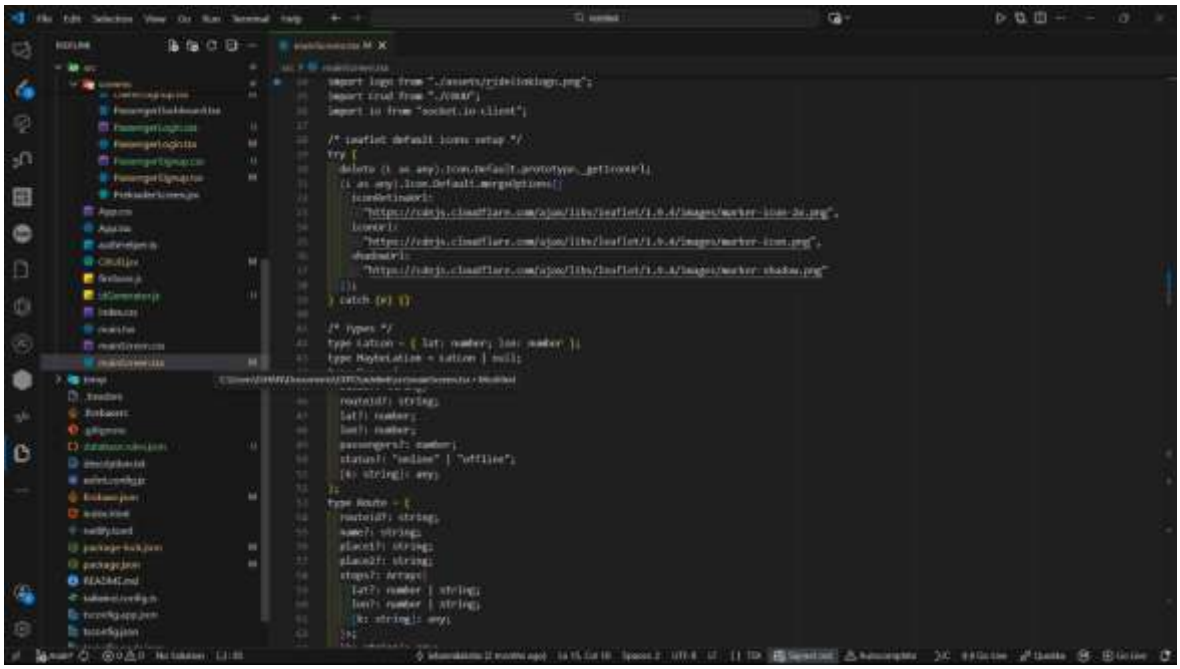

Figure 4.28 Main Screen Code

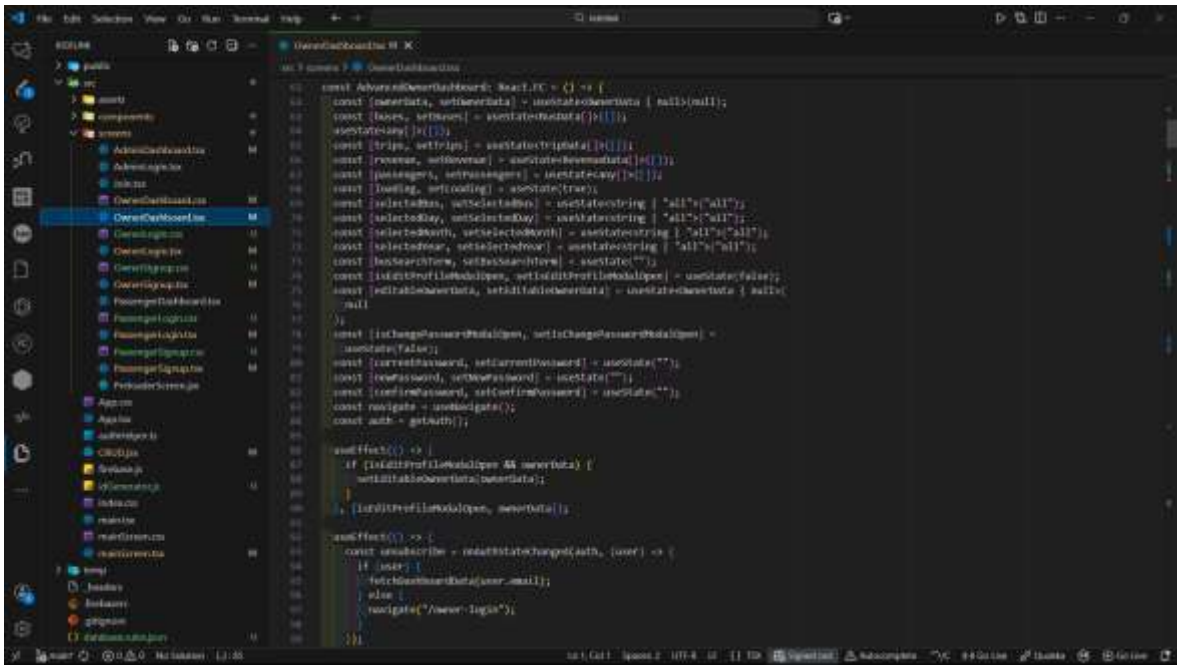

Figure 4.29 Owner Dashboard Code

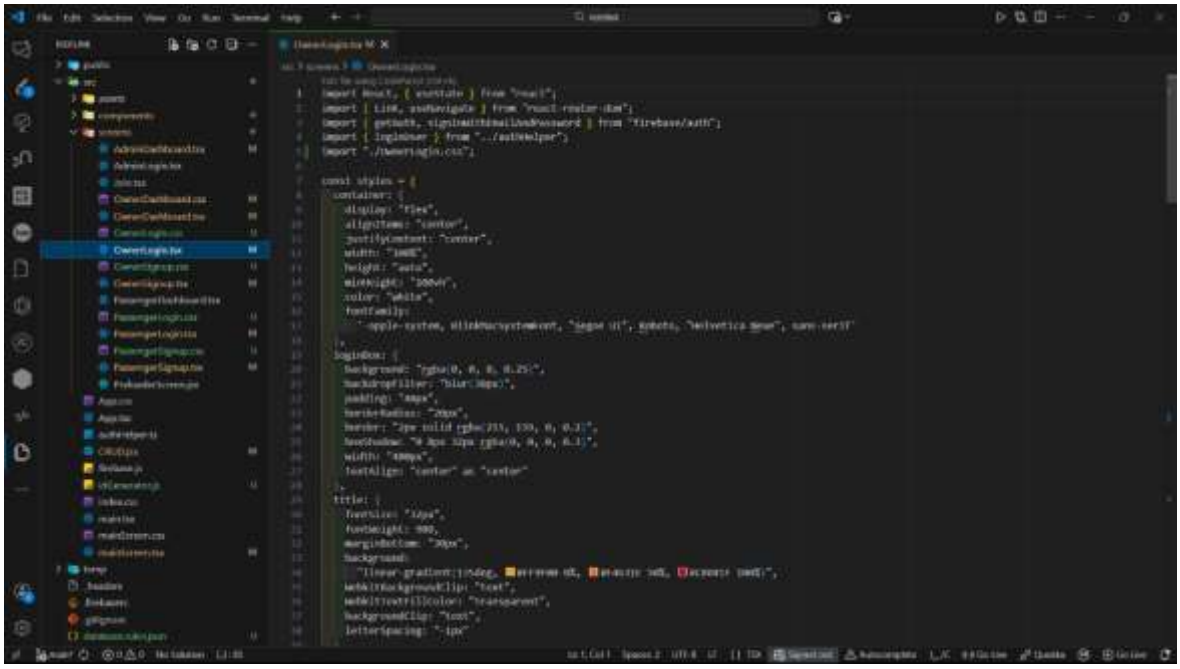

Figure 4.30 Owner Login Code

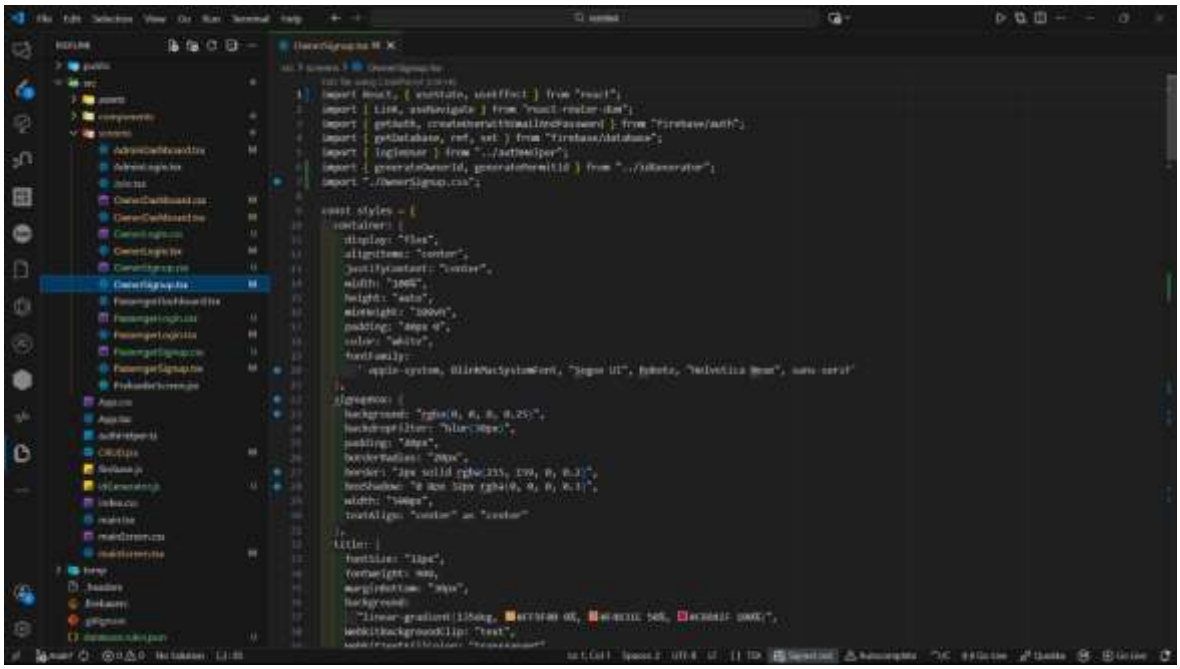

Figure 4.32 Owner Sign Up Code

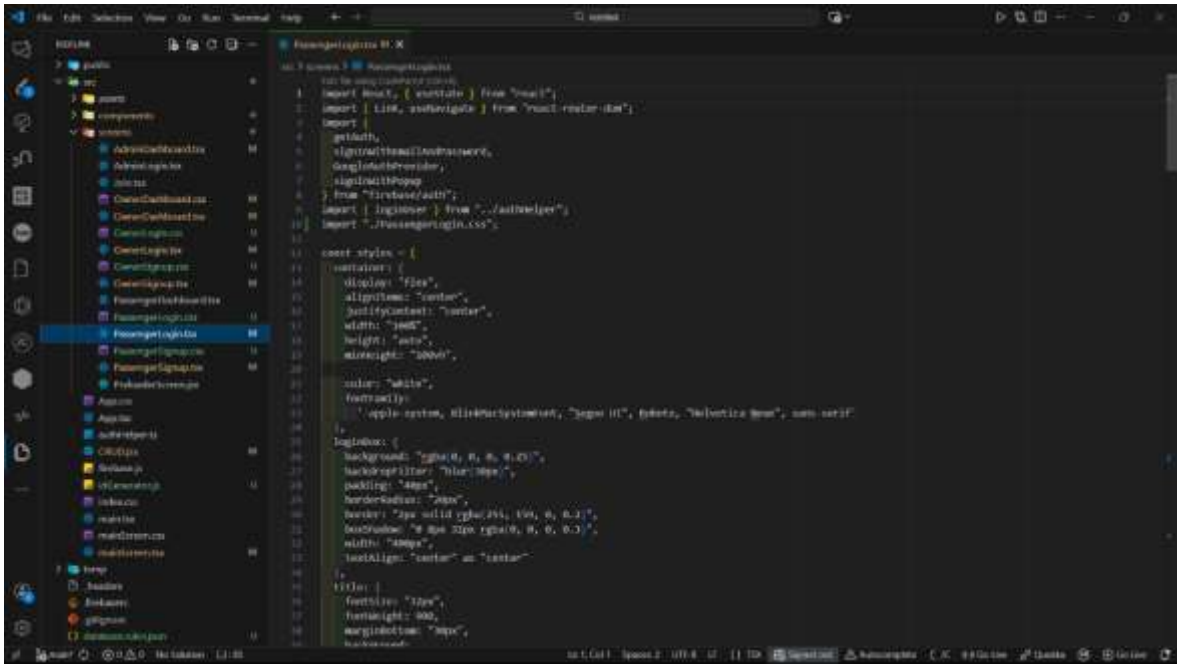

Figure 4.31 Passenger Login Code

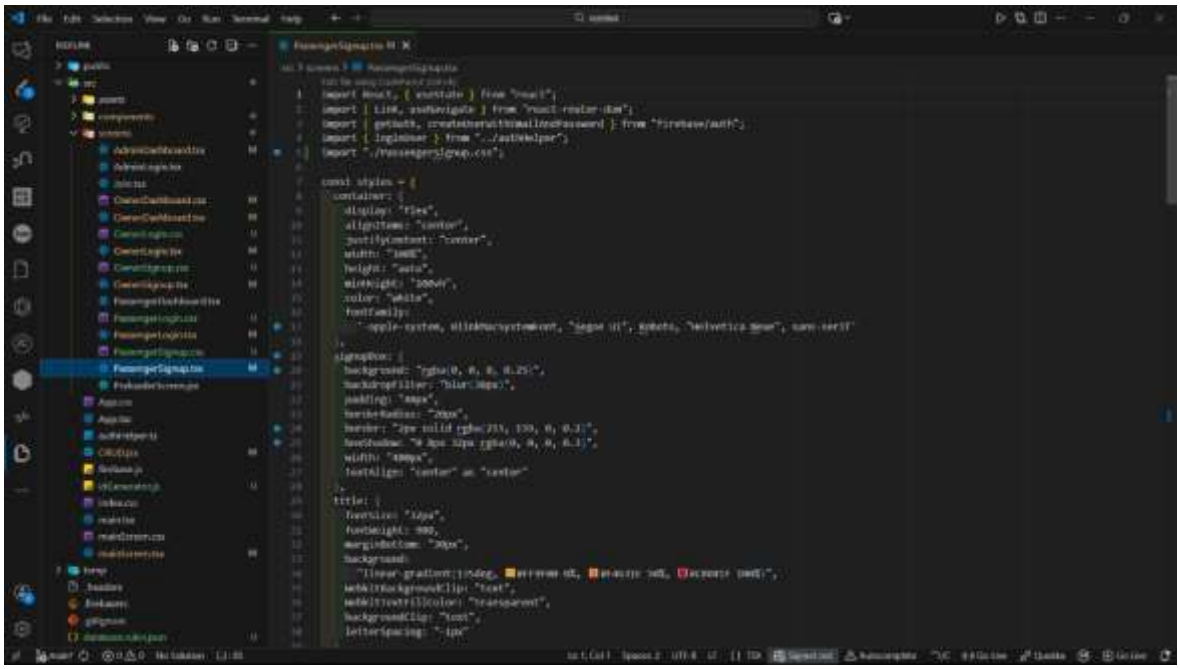

Figure 4.33 Passenger Sign Up Code

3 Arduino Code

Figure 4.34 Arduino Code

4 Node Backend

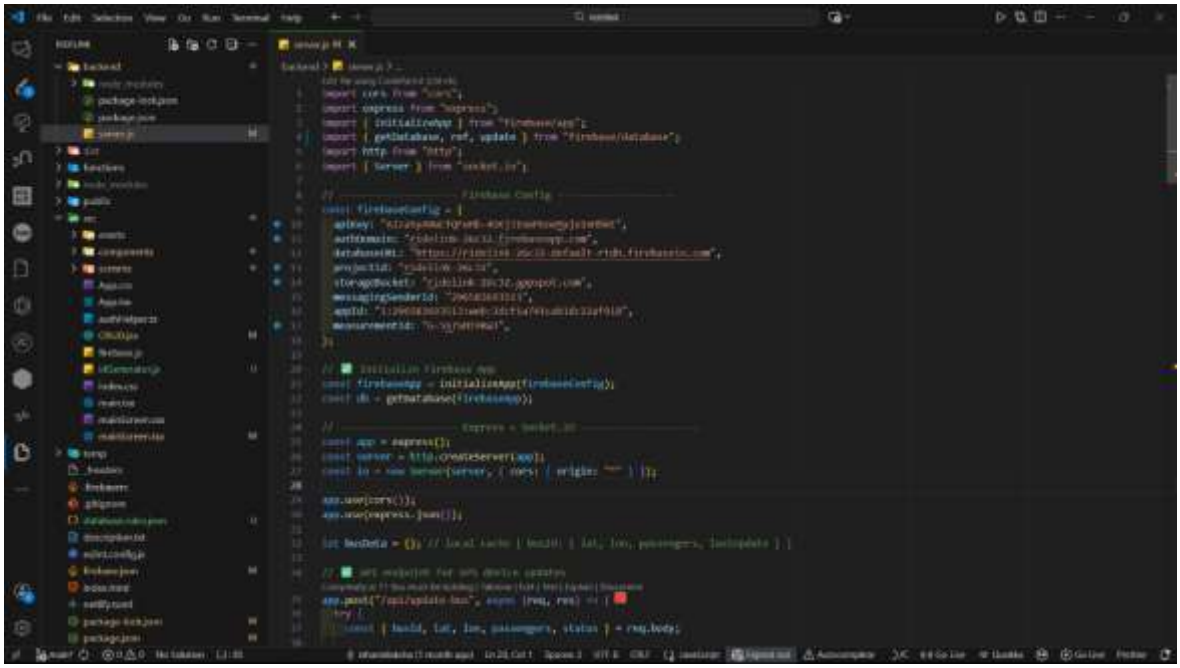

Figure 4.35 Backend Code

5 Firebase

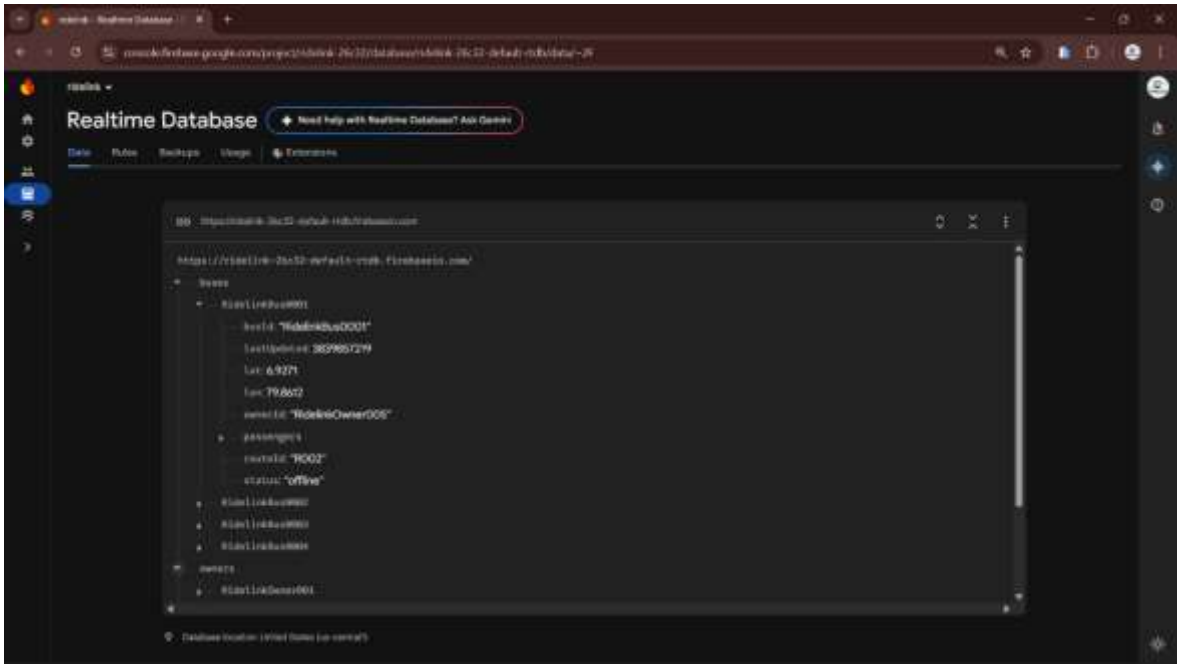

Figure 4.37 Realtime Database on Firebase Code